

KopiBridge AI

Multimodal Evidence Led Career Guidance

Final Project Report

CM3020 Artificial Intelligence

Project Idea 1 Section 4.1

Orchestrating AI models to achieve a goal

Code repository <https://github.com/aevto/KopiBridge-AI>

Existing deployment <https://kopi-bridge-ai.vercel.app>

28 September 2026

Abstract

KopiBridge AI helps early-career applicants compare resume evidence with a target role and prepare truthful next steps. Its final implementation orchestrates three distinct pretrained models: image-to-text extraction, text guidance and speech transcription. Scanned resume pages become editable text; a deterministic engine builds the authoritative evidence report; a guarded language model refines permitted guidance; and a transcribed interview answer receives feedback tied to that same report. The applicant reviews both extracted text and transcription before relying on them. The product includes authenticated accounts, private history, server-enforced analysis credits and print export. Evaluation combines executable database tests, browser integration checks and exploratory comparisons of six model configurations across synthetic inputs. Initial feedback quotation failures motivated a schema-constrained citation design, which passed all six subsequent quotation checks. These results establish integration feasibility, not hiring prediction accuracy or proven career benefit. Important limitations include heuristic negation errors, a small synthetic evaluation set, compressed evidence context and the absence of a participant study. The contribution is a coherent multimodal preparation workflow that keeps claim status inspectable rather than delegating it to generated prose.

Chapter text totals 8,466 words. Counts include subheadings and table content; exclude chapter titles, captions, title page, abstract and references. The total remains below 10,500 even if the abstract is included.

Chapter	Words	Maximum
Introduction	717	1,000
Literature Review	2036	2,500
Design	1418	2,000
Implementation	1982	2,500
Evaluation	1967	2,500
Conclusion	346	1,000
Total	8466	10,500

Contents

Introduction	4
Literature Review	6
Design	11
Implementation	17
Evaluation	49
Conclusion	56
References	57

1. Introduction (717/1000 words)

1.1 Project concept and motivation

KopiBridge AI is a resume-to-role gap analysis and career planning application for final-year computing students, recent graduates and early-career developers targeting AI-adjacent technical roles. Relevant roles include junior AI engineer, LLM application developer, AI solutions engineer, software engineer for AI products, technical solutions engineer and junior data or machine-learning positions. These roles commonly combine programming, data manipulation, web development, deployment, collaboration and emerging AI tools. A student may possess parts of this profile through coursework and personal projects but still struggle to decide whether a resume proves the requirements of one particular vacancy.

This problem is more complex than keyword matching. A job description may list Docker, cloud deployment or retrieval-augmented generation, while the applicant has never implemented those technologies. Telling the applicant to insert the missing terms would improve superficial overlap but could create an indefensible claim. Conversely, an applicant may already have relevant evidence but describe it using weak or unfamiliar language. The useful distinction is therefore between a wording problem and a proof problem. KopiBridge AI is motivated by the need to make that distinction explicit.

The product accepts resume text, either pasted directly or extracted from a PDF, and a target job description. It produces a structured private report containing an overall alignment estimate, category scores, existing strengths, critical gaps, an evidence checker, a prioritised action roadmap, resume wording guidance, interview preparation and a final application recommendation. The score is described as an estimate of document-to-role alignment, not a prediction of hiring success. Recommendations are constrained by three honesty labels: Safe to add now when the resume directly supports the claim; Reframe carefully when the underlying experience appears supported but should not be exaggerated; and Needs proof first when the capability should not be claimed until evidence has been created.

The project began with local-only analysis and subsequently added authenticated history, transactional credits and guarded text refinement. The final extension addresses the confirmed AI orchestration template through image, text and audio models working towards one career-preparation goal. Without an OpenAI key, pasted text and selectable PDFs still produce the complete deterministic report. Image reading and transcription require provider access; unavailable interview feedback falls back to an explicitly labelled checklist. These fallbacks preserve utility but are not represented as a successful three-model run.

1.2 Aim, objectives and research questions

The project aim is to help an early-career applicant turn a resume and target job description into an honest, evidence-led decision and a practical improvement plan.

The objectives are:

1. Design and implement a guided web workflow for resume input, job-description input, analysis and report review.
2. Develop an inspectable matching engine that distinguishes strong, partial, weak and missing evidence.
3. Generate specific proof-building recommendations without inventing skills, responsibilities or measured outcomes.
4. Protect personal reports through authenticated sessions, server-side authorisation and owner-only database policies.
5. Enforce a daily usage allowance correctly under duplicate, concurrent and failed requests.
6. Provide private history, deletion and a print-to-PDF report suitable for repeated career planning.
7. Evaluate the integrated software and each model with reproducible fixtures, component-appropriate measures and explicit limitations.

Three questions guide evaluation: (RQ1) Can distinct image, text and audio models form a coherent resume-to-interview workflow? (RQ2) Can deterministic evidence labels and grounded quotations prevent unsupported changes while allowing useful guidance? (RQ3) Do authentication, ownership, credits and recovery controls work across the integrated path?

1.3 Scope and project template

The confirmed template is CM3020 Artificial Intelligence, Project Idea 1, section 4.1: Orchestrating AI models to achieve a goal. Its final-product requirement is at least three pretrained models operating on different data types or domains, combined in a purposeful working system. KopiBridge AI uses gpt-4.1-mini for resume images, gpt-5.6-terra for text guidance and gpt-4o-mini-transcribe for interview audio. PDF.js and the deterministic matching engine are supporting software, not additional pretrained models. The same saved report connects the three model roles, rather than presenting independent demonstrations.

The final scope includes selectable and scanned PDFs, image uploads, editable extracted text, local evidence analysis, guarded narrative guidance, short audio answers, reviewed transcripts and private interview feedback. Employer-side selection, speech-based personality assessment, automated applications, public reports and LLM-generated scores are excluded. The existing text product is deployed; the multimodal extension evaluated here runs locally against live providers and the approved hosted database migration. It has not been claimed as a newly deployed production release.

2. Literature Review (2036/2500 words)

2.1 Automated recruitment and the applicant perspective

Digital systems influence recruitment through advertising, sourcing, screening, assessment and ranking. Bogen and Rieke (2018) argue that predictive hiring technologies can reproduce institutional and structural bias and that numerical rankings may overstate small differences between applicants. Raghavan et al. (2020) similarly find a gap between vendors' fairness claims and the evidence or practices used to support them. These studies concern employer-side systems, whereas KopiBridge AI is an applicant-side support tool and does not recommend hiring decisions. Nevertheless, they expose two design risks that remain relevant: presenting a score with more authority than its method deserves, and concealing value judgements inside an opaque model.

KopiBridge AI addresses the first risk by labelling the overall result as an alignment estimate and explaining that it is not a prediction of hiring success. It addresses the second through requirement-level evidence and visible category weights. This does not make the method neutral. A curated skill list still encodes assumptions about which capabilities matter, and repeated terms are not equivalent to competence. Transparency therefore supports critique rather than proving fairness. The applicant-side orientation reduces the direct harm of automated rejection but can still influence whether a user applies, edits a resume or delays an application. Conservative language and contestable evidence are consequently necessary.

Commercial applicant tools often emphasise applicant-tracking-system compatibility, keyword coverage or document presentation. Those functions may help users communicate experience, but they can also collapse employability into textual similarity. KopiBridge AI adopts a different unit of value: defensible evidence. A missing requirement becomes either a reasonable optional advantage, a positioning issue, or a task through which proof can be created. This reframes optimisation as capability development rather than gaming an assumed screening mechanism.

2.2 Skills representation and resume-to-job matching

Occupational frameworks illustrate why role matching needs structure. ESCO organises occupations, skills, competences and qualifications, while the *ONET Content Model separates worker characteristics, worker requirements, experience requirements, occupational requirements and occupation-specific information* (European Commission, n.d.; National Center for ONET Development, n.d.). Both are substantially broader than the project requires, but they challenge an undifferentiated bag-of-keywords approach. A bachelor's degree, Python, stakeholder communication and deployment evidence are different kinds of requirements and should not be presented as interchangeable matches.

The current engine therefore uses a deliberately smaller taxonomy for junior AI-adjacent roles. Signals are grouped into technical skills, experience, tools, education and communication. Each signal contains aliases, adjacent terms, a category, a weight and an evidence-building action. The narrower taxonomy is easier to inspect and maintain, but has reduced coverage compared with ESCO or O*NET. It may omit a niche technology or interpret equivalent phrases

inconsistently. The design accepts this trade-off for the present stage because explainability and reliable testing are more important than broad occupational coverage.

Contextual language models demonstrate how future matching could move beyond exact wording. BERT models tokens using bidirectional context and achieved strong results across multiple language-understanding tasks (Devlin et al., 2019). Sentence-BERT creates sentence embeddings that can be compared efficiently for semantic similarity (Reimers and Gurevych, 2019). In principle, embeddings could identify that “built a REST service” and “backend API development” express related experience. Such methods might improve recall where a rule-based matcher misses paraphrases.

Semantic similarity is not identical to evidential support, however. Two sentences may be close in meaning without establishing the depth, recency or context requested by an employer. An embedding could also create a false positive that is difficult for a user to challenge. For a career tool, false confidence may be more harmful than an obvious omission because it can encourage unsupported resume claims or weak interview preparation. The current engine consequently prefers conservative aliases and exposes matched snippets. A later hybrid approach could use semantic retrieval to propose candidate evidence while retaining deterministic thresholds, user confirmation and the three honesty labels.

2.3 Explainability, trust and evidence-calibrated guidance

Explainability literature distinguishes between producing an explanation and evaluating whether that explanation is appropriate for its audience and task. Guidotti et al. (2018) survey rule, feature, example and surrogate approaches for explaining black-box models. Doshi-Velez and Kim (2017) argue that interpretability lacks a single agreed definition and should be assessed through application-grounded or human-grounded evaluation when possible. This is important for KopiBridge AI because a technically traceable score is not automatically useful. A student needs to understand what was detected, why a gap matters and what to do next.

The evidence map operationalises explanation at requirement level. Strong, partial, weak and missing labels describe the amount of resume support, while Safe to add now, Reframe carefully and Needs proof first describe what action is ethically defensible. These are related but distinct dimensions: evidence strength describes the analysis; claim status governs the recommendation. For example, direct evidence of a deployed Next.js application may support clearer wording now, while an absent Docker signal should lead to a containerisation task rather than a rewritten claim.

This approach responds to two trust problems. First, a single score compresses heterogeneous evidence and can create false precision. Category scores and evidence snippets let the user inspect the composition of that score. Second, generative systems can produce fluent but unsupported advice. The local engine creates structured output from known rules, and resume rewrites are prohibited from inventing metrics or responsibilities. Fluency is deliberately secondary to grounding. The limitation is that rules can be rigid, and a polished interface may still make the output seem more authoritative than warranted. Repeated disclaimers alone are insufficient; future user testing must examine whether participants can accurately explain what the score does and does not mean.

2.4 Actionable career guidance and proof-building

Identifying a gap does not establish a useful intervention. Advice such as “improve cloud skills” leaves the learner to infer scope, sequence and evidence. KopiBridge AI instead expresses high-priority gaps as bounded artefacts: deploy one working project, add a Dockerfile and setup instructions, implement one test and error-handling path, or build a small retrieval demo only when retrieval is relevant to the role. This turns the report into a plan for producing observable evidence.

The roadmap is organised across today, this week, this month and longer term, while a complementary four-week view supports the original 30-day prototype. Short-term tasks improve the resume and audit existing skills; medium tasks build or strengthen one proof project; later tasks focus on deployment, documentation, testing, measurable evidence and application preparation. The design recognises that a student cannot manufacture professional tenure in thirty days. It therefore distinguishes a genuine experience gap from a project-sized technical gap and avoids implying that every requirement is mandatory.

Actionability also depends on effort and priority. Critical gaps include why the requirement matters, an estimated effort and a concrete next step. Optional advantages are kept below core requirements. Communication is not automatically assigned roadmap space when the resume already demonstrates it. This preserves limited attention for the highest-value actions. The main unresolved question is whether generated tasks are feasible for users with different schedules and levels of confidence. A longitudinal re-analysis after task completion would provide stronger evidence than a one-session satisfaction rating.

2.5 Human-centred design and usability evaluation

ISO 9241-210 describes human-centred design as an iterative process grounded in users, tasks and context across the system life cycle (ISO, 2019). Nielsen's heuristics provide more operational criteria, including visibility of system status, error prevention, user control, recognition rather than recall and aesthetic minimalist design (Nielsen, 1994). These principles suit a workflow in which users provide sensitive text, spend a limited credit and interpret a detailed report.

The interface applies these ideas through a guided three-stage input, visible credit status, editable role detection, validation before credit reservation, loading progress, recoverable errors and a report ordered by decision value. The top section answers the questions a user is most likely to ask first: how close is the document, what is already strong, what are the top gaps, and what should happen next. Detailed matches and resume improvements appear later to avoid a dense administrative dashboard. Mobile behaviour and print export are treated as core states rather than optional styling.

The System Usability Scale (SUS) offers a lightweight standardised questionnaire after task completion (Brooke, 1996), but SUS alone would not establish that recommendations are honest or understood. A suitable evaluation should combine task success, time, observed errors, comprehension questions, SUS and qualitative interviews. Example comprehension checks should ask participants whether a “Needs proof first” item belongs on the current resume

and whether the overall percentage predicts recruitment success. This links interface quality directly to the project's ethical goals.

2.6 Privacy, security and trust boundaries

Resume documents and interview answers can contain personal information. Selectable PDF text is extracted locally; scanned pages and audio require explicit consent before provider processing. KopiBridge does not persist those raw uploads. Reports retain short evidence excerpts and reviewed interview answers, so they remain personal data even though complete original documents are not stored. Deleting a report cascades to its interview feedback, while minimal usage events remain for quota enforcement. Provider retention is separate: disabling retrievable response storage is not a guarantee of zero provider retention.

Authentication and ownership are also relevant to trust. Supabase's server-side rendering guidance uses cookie-based sessions so server components and route handlers can resolve the current user (Supabase, n.d.-a). Row-level security policies can use `auth.uid()` to constrain rows to the authenticated owner (Supabase, n.d.-b). KopiBridge AI combines route-level checks with database policies so a client-supplied user identifier is never accepted as authority. OWASP recommends validating input from untrusted sources as early as possible (OWASP, n.d.); the analysis endpoint uses schema validation and limits before reserving a credit.

These controls reduce common failures but do not eliminate operational risk. Email delivery depends on external provider configuration, authentication libraries evolve, and database functions declared with elevated privileges require careful grants and fixed search paths. Security testing must therefore include direct endpoint calls and cross-user access, not only navigation through the interface.

2.7 Multimodal extraction and human review

Visual document understanding differs from parsing an existing text layer. Kim et al. (2022) introduce Donut as an OCR-free document-understanding transformer and identify costs and error propagation in OCR-dependent pipelines. This motivates evaluating page images as model inputs, but Donut was not implemented or benchmarked here. A hosted vision model was chosen to minimise separate infrastructure and use the existing provider boundary. The trade-off is external processing, variable cost and potential hallucinated text. A conventional local OCR engine remains a useful future baseline, not a tested rejected alternative.

Radford et al. (2023) demonstrate broad speech-recognition capability from large-scale weak supervision. Their benchmark results justify considering pretrained transcription, not assuming it will accurately capture every applicant or technical term. KopiBridge compares two hosted transcription configurations using word error rate and latency, then requires user correction. Interview content is assessed as an unverified statement; neither accent nor vocal style is used to infer personality or employability. Synthetic speech is suitable for regression fixtures but insufficient for claims about accent fairness or real interview performance.

Amershi et al. (2019) emphasise that human-AI interaction must accommodate imperfect predictions and support correction. The editable extraction and transcription stages apply that principle directly. OpenAI structured outputs constrain response shape (OpenAI, n.d.-c), but shape compliance alone cannot prove factual grounding. KopiBridge therefore combines schema validation with source-excerpt constraints and preserves deterministic claim labels. The model comparison must evaluate these application-specific properties, not only fluency. This synthesis supports separate measures for transcription fidelity, evidence invariants and user comprehension.

2.8 Synthesis and project position

The literature identifies a design space rather than a single solution. Recruitment research warns against opaque rankings and inflated validity claims. Occupational models support structured categories. NLP research offers better semantic recall but introduces explanation and calibration challenges. Interpretability work shifts attention from the existence of reasons to their usefulness for a specific user. Human-centred design provides an evaluation framework, while security guidance establishes that sensitive, authenticated workflows require server-side trust boundaries.

KopiBridge AI combines applicant-side evidence analysis with a multimodal preparation workflow. Its contribution is integration and constraint design, not training a new foundation model. The relevant test is whether pretrained capabilities improve access and preparation while keeping evidence, ownership and uncertainty visible. The following implementation and evaluation therefore distinguish software invariants, extraction accuracy, quotation grounding and user usefulness.

3. Design (1418/2000 words)

3.1 Users, needs and context

The primary persona is a final-year computing student or recent graduate applying for an AI-adjacent technical role. The user may have coursework, projects, hackathons or limited internship experience, but may not understand how those experiences map to a specific vacancy. They need a quick initial decision, detailed evidence they can inspect, and tasks that fit an early-career portfolio rather than advice that assumes access to production systems or professional authority.

A representative user uploads a selectable PDF or scanned resume, reviews the extracted text, pastes a vacancy and confirms its role metadata. After generating a private report, the user reviews the most important gaps and chooses one interview question. A short recorded or uploaded answer is transcribed, corrected and submitted for feedback grounded in that report. The applicant can revisit the combined result or print it, and later repeat analysis after producing genuine new evidence.

3.2 Requirements and design priorities

The principal functional requirements are summarised in Table 1.

Area	Required behaviour	Design rationale
Identity	Sign up, login, persistent session, reset and logout	Private history requires a stable authenticated owner
Input	PDF/image, pasted text, role metadata and reviewed audio answer	Supports realistic and reliable demo paths
Analysis	Structured deterministic report with grounded evidence	Preserves inspectability and works without an LLM
Credits	Three accepted analyses per Singapore day	Controls usage while remaining simple for students
History	Owner-only list, detail and deletion	Supports reflection without public report links
Export	Browser print dialog and print-specific layout	Produces a portable report without a second rendering service

Table 1. Principal functional requirements and design rationale.

Non-functional priorities are security, correctness, privacy, accessibility, responsiveness, maintainability and deployment reliability. Inputs must be treated as untrusted; validation failures must not consume credits; duplicate or concurrent submissions must not double-spend; internal

failures must restore a reservation; and one user must never access another user's report. The product must also explain limitations without interrupting the core task.

3.3 Information architecture and interaction flow

Public pages explain the value before requesting registration. Authenticated navigation is deliberately small: Dashboard, New Analysis, History and Settings. The dashboard combines remaining credits, a primary action and recent reports. The analysis workspace uses three visible stages: resume evidence, target role and confirmation. The report detail page doubles as the saved-history view, avoiding duplicate report implementations.

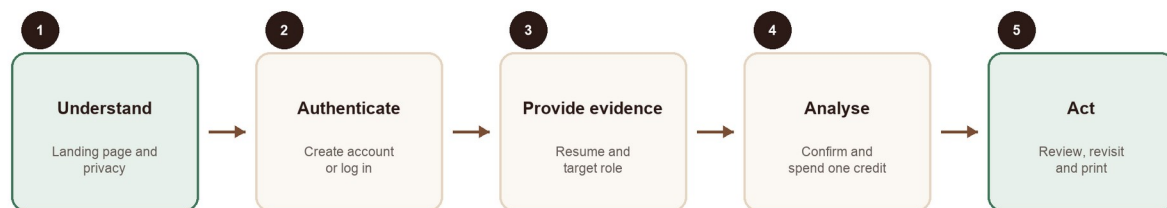


Figure 1. Primary user flow from public entry to a privately stored and printable report.

System status is visible throughout. Credit balance and reset policy appear before submission. The analysis button is disabled until required fields meet validation conditions. Loading communicates that the report is being built and duplicate submission is prevented. Recoverable network errors preserve the entered text. Zero credits produces a specific next-reset state instead of a generic error.

3.4 System architecture

The architecture separates presentation, trusted server orchestration, deterministic domain logic, guarded language refinement and persistence. Next.js App Router pages and React components provide the public and authenticated interfaces. Browser code extracts PDF text and sends validated text to a Next.js route handler. The route resolves the trusted server session, reserves a credit, runs the local TypeScript engine first and constructs a complete structured report. When OpenAI is configured, a constrained server-only step receives bounded, redacted context and returns schema-validated narrative fields; a whitelist merge prevents it from changing scores, requirement evidence, gap severity or honesty labels. Failure returns the local result. Supabase Auth manages identity, while PostgreSQL row-level security and functions protect ownership and credit invariants.

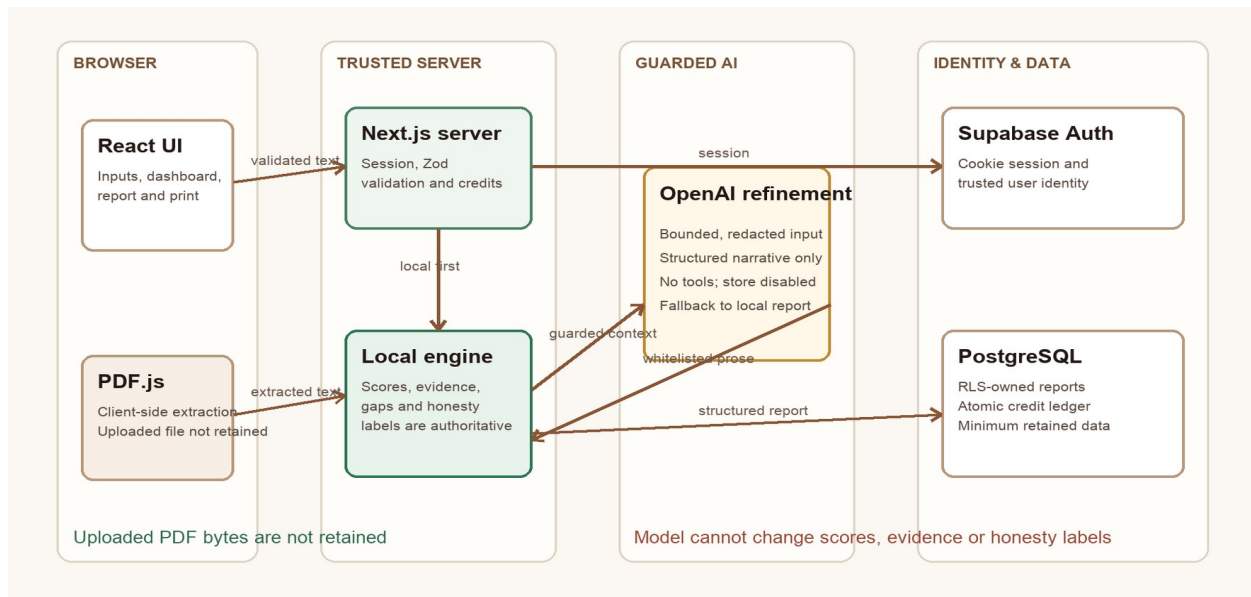


Figure 2. KopiBridge AI component and trust-boundary architecture.

Raw PDFs, images and audio are not persisted by KopiBridge. History contains report JSON and role metadata. Interview practice adds the selected question, reviewed transcript, structured feedback, source, model and timestamp. Audit records hold operation identifiers and Singapore usage dates, not raw media. These are deliberate retention boundaries, not a claim that the stored report is anonymous.

3.5 Analysis and report design

The authoritative domain result is a typed AnalysisResult, not free-form HTML or unconstrained model output. Requirements are selected from a curated signal catalogue based on the job description. Resume evidence produces a strength label and numeric contribution. Category weighting creates an overall estimate; gaps are ranked by severity and requirement importance; and action templates are specialised according to the gap. Resume wording guidance follows the same evidence policy. The optional OpenAI response is also schema-validated and can only refine permitted narrative fields before the final AnalysisResult is stored.

The report hierarchy reflects the ten-second comprehension goal. The fit estimate, role-fit explanation, existing strengths, critical gaps, priority actions and final recommendation appear before deeper evidence. The evidence checker is visually prominent because it supports safe decisions. The action roadmap and four-week proof plan translate gaps into concrete deliverables, followed by resume wording, interview preparation, score breakdown and requirement details. This is progressive disclosure through ordering rather than hidden navigation.

The visual system uses warm off-white page surfaces, white content areas, espresso typography, restrained coffee and moss accents, amber for partial evidence and clay red for higher risk. Cards use modest radii, light borders and controlled shadows. Typography and

spacing create hierarchy without decorative effects associated with generic AI dashboards. The same semantic colours appear in badges, progress bars, empty states and print output.

3.6 Data and security design

Five tables support the final workflow: `analyses`, `daily_credit_balances`, `credit_transactions`, `model_operations` and `interview_practice`. The original report and credit structures are preserved. New model-operation records enforce six daily attempts separately for vision, transcription and interview feedback; practice records reference both an owned report and accepted operation. Row-level security restricts reads and deletions to the owner. Raw upload storage is absent.

Credit operations are exposed through narrowly granted security-definer functions. `reserve_analysis_credit` derives the owner from `auth.uid()`, locks the daily balance row, rejects a fourth use and records a unique reservation. `store_completed_analysis` writes the report against the matching reservation. `refund_analysis_credit` restores the balance and records a refund when processing fails. The API never accepts a client user ID. The idempotency key prevents a repeated request from creating a second deduction, while row locking serialises concurrent reservations.

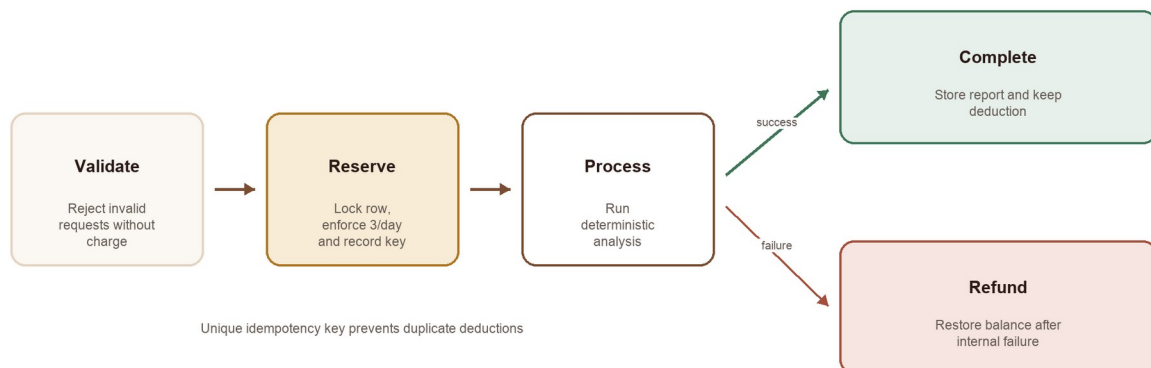


Figure 3. Atomic credit reservation, completion and refund states.

3.7 Workplan and feasibility

Phase	Main work	Evidence of completion or next output
Foundation	Local analysis, PDF extraction, report and print	Implemented Stage 1 prototype and preliminary evaluation
Productisation	Auth, credits, private history, validation, responsive product UI	Implemented and deployed with migrations and tests
Evaluation	Automated checks, security scenarios, heuristic review	Automated and authenticated QA completed; user study pending
Final refinement	User study, signal tuning, accessibility and report revision	Planned from evaluation findings
Language refinement	Guarded OpenAI narrative refinement after deterministic analysis	Implemented; cannot change scores, evidence, severity or honesty labels
Multimodal integration	Vision extraction, transcription, grounded feedback	Implemented and locally tested; migration applied
Submission	Final report, screenshots, comparison and limitations	Measured evidence; deployment still separate

Table 2. Project workplan and current status.

The workplan progressed from the local engine to product infrastructure, guarded text refinement and multimodal integration. The approved additive migration and local integration tests complete the implemented scope. Deployment of the new extension, representative human evaluation and stronger provider-independent validation remain distinct next steps. No unrecorded tutor feedback or participant study is assumed; the revised scope follows the confirmed section 4.1 template and observed test failures.

3.8 Ethical considerations

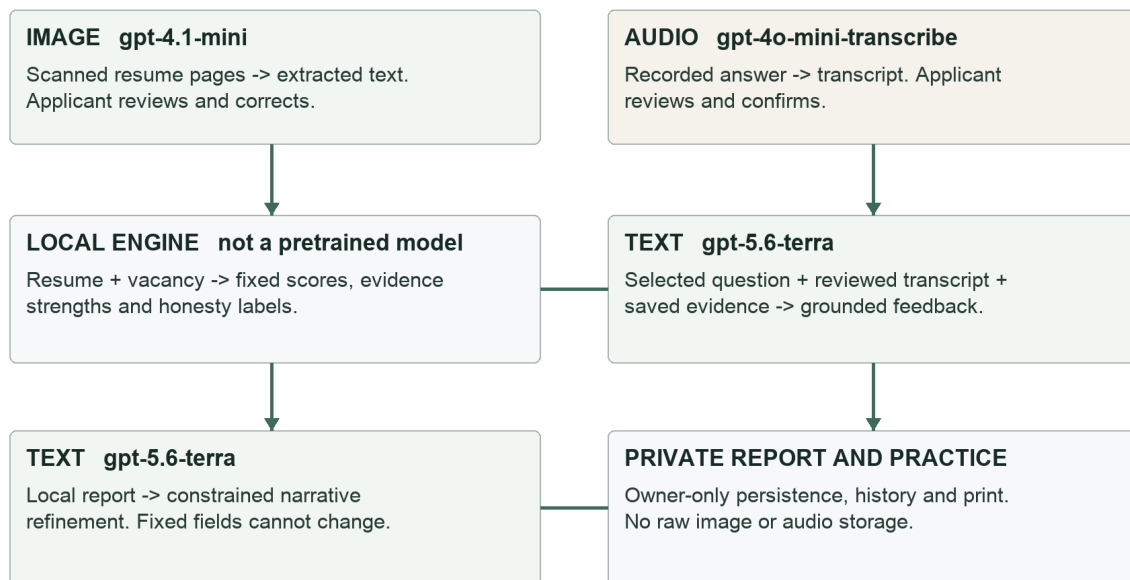
The product is designed to support, not replace, user judgement. It does not rank people for employers, guarantee outcomes or infer protected characteristics. Scores are estimates of textual and evidential alignment. Users can edit inputs, inspect evidence and delete stored reports. Recommendations explicitly discourage bluffing and invented metrics.

Bias can still enter through the signal catalogue, category weights and choice of “typical” junior roles. English-language resumes and conventional project evidence are privileged by the current design. A future review should include diverse resume formats, educational pathways and communication styles. Clear limitations, conservative matching and user control reduce harm but do not remove the need for empirical evaluation.

3.9 Integrated model orchestration

Figure 4 shows the final information flow. The vision model supplies editable text rather than an employment judgement. The text model refines a precomputed report, and the audio model supplies a transcript rather than a score. The reviewed transcript and selected question then return to the text model with the same report evidence. This dependency makes the modalities serve one goal: turning an existing application into a defensible account of readiness.

Each media endpoint authenticates independently, checks ownership where a report is involved, validates payload bounds and reserves an operation. An advisory transaction lock serialises each user and operation kind; a unique user-kind-request key prevents repeated charging of the same operation. Six attempts per kind per Singapore day are distinct from three full analyses. Failed media attempts count towards the limit to prevent unlimited provider retries. This policy is disclosed before submission and does not refund or alter full-analysis credits.



Server boundary: session checks | bounded inputs | consent | usage reservations | no client secrets

Figure 4. Final three-model workflow and its deterministic evidence boundary.

4. Implementation (1982/2500 words)

4.1 Technology stack and project structure

KopiBridge AI is implemented in TypeScript with Next.js 16 and React 19. Tailwind CSS supplies utility styling, PDF.js extracts text from uploaded PDFs, Zod validates analysis requests and model responses, Lucide provides interface icons, and the OpenAI SDK supports guarded narrative refinement. Supabase contributes email/password authentication, server-side session support and hosted PostgreSQL. Vitest covers deterministic logic, model guardrails and migration contracts; Playwright covers browser flows. The application is deployed on Vercel and remains a single Next.js codebase rather than being divided into premature services.

The App Router divides public, authentication and protected product routes. Shared components implement the branded shell, inputs, credit status, history and report. Domain modules under lib/ contain analysis, scoring, validation, authentication, guarded OpenAI refinement, history and credit behaviour. This separation keeps the deterministic engine independent from the report interface, provider and database. The types/analysis.ts contract defines every report section, allowing server, database and UI code to exchange structured data rather than generated markup.

4.2 Resume input and PDF extraction

ResumeUpload accepts PDF, PNG, JPEG and WebP with an 8MB client limit. PDF.js first attempts selectable-text extraction for up to twenty pages. A low-text PDF offers consented image reading for at most three pages. Pages are rasterised in-browser and bounded before transmission. The server validates format signatures, payload length and consent, then calls the vision model. Extracted text remains editable and warnings are shown. An upload control is disabled until hydration so an early selection cannot be lost before its event handler is ready.

This retains the low-cost selectable-PDF path while adding a meaningful vision requirement. Image requests have a 3.8MB server-body limit and are read incrementally, including when Content-Length is absent. Raw file data is transient. A sample and pasted text remain alternatives when extraction fails. Selecting a new valid file clears the prior text to avoid accidentally analysing a different resume; a failed analysis request, by contrast, preserves the current input.

4.3 Deterministic analysis engine

The core function has a stable typed boundary:

```
analyseResumeAgainstJob(  
  resumeText: string,  
  jobDescription: string,  
): AnalysisResult
```

The engine normalises text, detects job requirements from the curated signal catalogue and supplies a conservative default set if a broad description yields too few signals. For each relevant signal it counts direct aliases and related terms in the resume and captures a supporting snippet where possible. Evidence is mapped to Strong, Partial, Weak or Missing. The corresponding contribution values are 100, 72, 35 and 0. These values are heuristics rather than learned probabilities.

Category scores are weighted by the relevant requirements within each category. The final estimate combines skills at 35%, experience at 26%, tools at 20%, education at 12% and communication at 7%, redistributing effectively when no requirement exists in a category. A score at or above 82 is labelled Strong Match, 68-81 Good Match, 50-67 Developing Match and below 50 Early Match. The interface always presents the result as resume-to-role alignment.

Gap construction separates absence, weak evidence, experience deficits, positioning problems and optional advantages. Missing items become high priority, weak items medium and partial items low unless another rule raises their importance. Priority actions take the highest-weighted gaps and attach estimated effort and proof-building language. Roadmap generation then uses the actual gap identifiers: Docker creates a containerisation task, deployment creates a live deployment task, testing creates an implemented test or error-handling task, and RAG creates an optional small retrieval demo only when the target description requires it.

Resume recommendations do not interpolate invented achievements. Existing evidence can be marked Safe to add now, adjacent experience becomes Reframe carefully, and absent evidence becomes Needs proof first. Suggested bullets preserve the original experience and explicitly request a verified result where a measurement would strengthen the statement. Interview questions are generated from the target role and highest-priority gaps, including items the user should not bluff about.

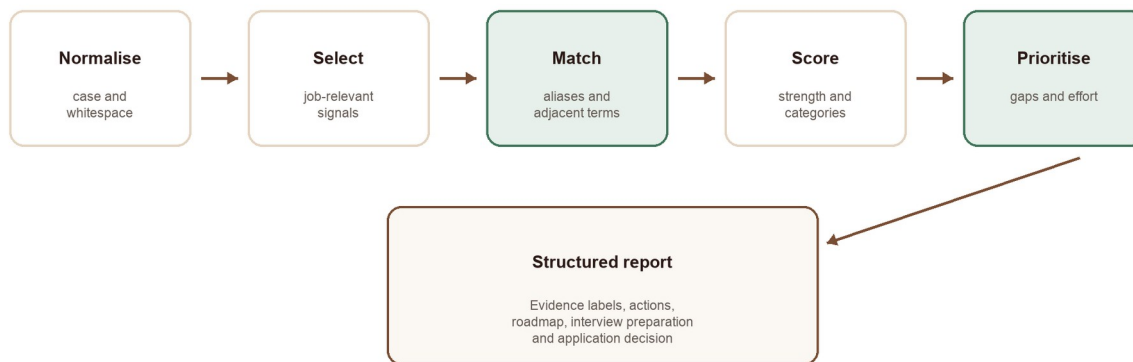


Figure 5. Deterministic transformation from untrusted text to a structured evidence-led report.

4.4 Guarded OpenAI guidance layer

The analysis route always produces the deterministic report before requesting language refinement. The server uses the OpenAI Responses API with a structured Zod output contract

and a configurable model whose current default is gpt-5.6-terra. Resume and job-description text are treated as untrusted data, normalised, length-bounded and redacted before use. The prompt exposes no secrets, enables no tools and requests store: false. The API key remains server-side and is never included in the client bundle.

The merge is deliberately asymmetric. Named narrative fields, such as the role-fit explanation, proof-building actions, roadmap wording, resume guidance and interview preparation, may be improved. Deterministic scores, evidence snippets and strength labels, matched requirements, gap identifiers and severity, and the Safe to add now / Reframe carefully / Needs proof first classifications are copied from the local result. A schema error, timeout or provider failure leaves the local report intact and triggers the existing credit refund only when the overall analysis cannot be completed. This design gains clearer guidance without transferring truth control to the model.

4.5 Authentication and route protection

Authentication uses Supabase Auth with cookie-based server-side sessions. Separate browser and server clients prevent secret material from entering client bundles. A proxy refreshes session claims and propagates updated cookies. Protected server components call `requireUser`, which redirects a visitor to `/login` with the intended destination. Login and signup validate email and password locally, show provider-safe errors and return the user to the requested protected route. Password reset and update pages use the provider's supported email flow.

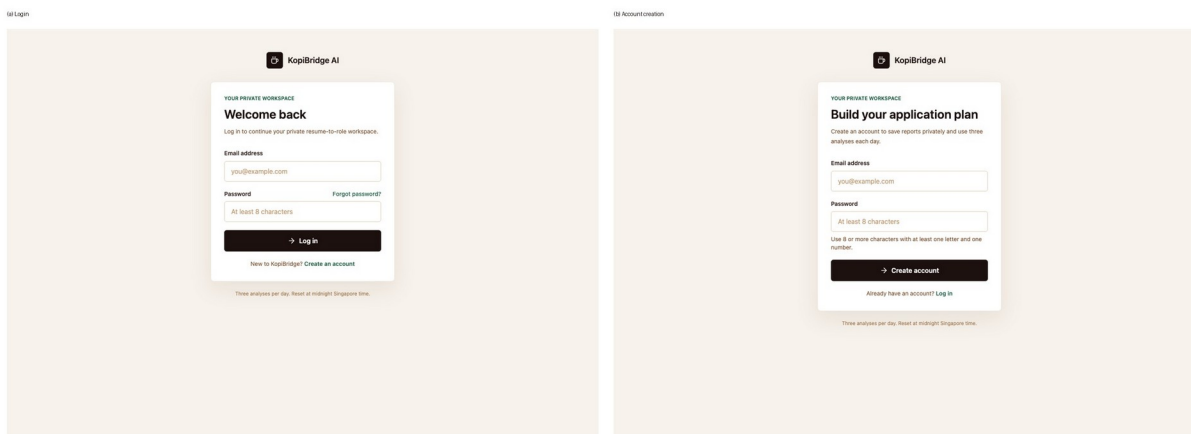


Figure 6. Branded login and account-creation interfaces with visible validation expectations. Earlier text-product build.

API handlers independently resolve the current session, so a caller cannot bypass protection by calling the endpoint directly. The application does not accept a user ID in the analysis schema. History queries include the trusted user ID, while row-level security repeats the owner check at the database boundary. This defence in depth is important because client navigation is a convenience, not an authorisation control.

4.6 Atomic credits and idempotent analysis

One accepted analysis consumes one of three credits for the current Asia/Singapore date. Input is validated before reservation. The route then calls `reserve_analysis_credit` with a client-generated idempotency key. The PostgreSQL function derives the owner from `auth.uid()`, creates or locks the daily balance using `FOR UPDATE`, checks the limit and inserts a unique ledger transaction. Replaying the same idempotency key returns the existing reservation rather than spending again.

After reservation, the route runs the deterministic engine. `store_completed_analysis` verifies the reservation owner and stores a completed report in the same trusted database layer. If analysis or persistence throws an internal error, a catch path calls `refund_analysis_credit`, decreases the used count without going below zero and records a `failed_analysis_refund`. Authentication, invalid input and zero-credit rejection occur before processing and are not charged.

The transaction ledger is more extensible than a client-side counter. Event types can later represent manual, promotional or purchased adjustments, while the current balance remains fast to read. Database row locking and uniqueness constraints are essential: disabling a button alone could not prevent concurrent browser tabs or direct API calls from overspending.

4.7 History, privacy and report export

Successful reports are automatically stored in the `analyses` table. The history list retrieves summary fields, and the detail route retrieves the structured JSON report only for its owner. Deletion asks for confirmation and sends an authenticated `DELETE` request; a missing or foreign identifier returns 404 so the API does not reveal whether another user's report exists.

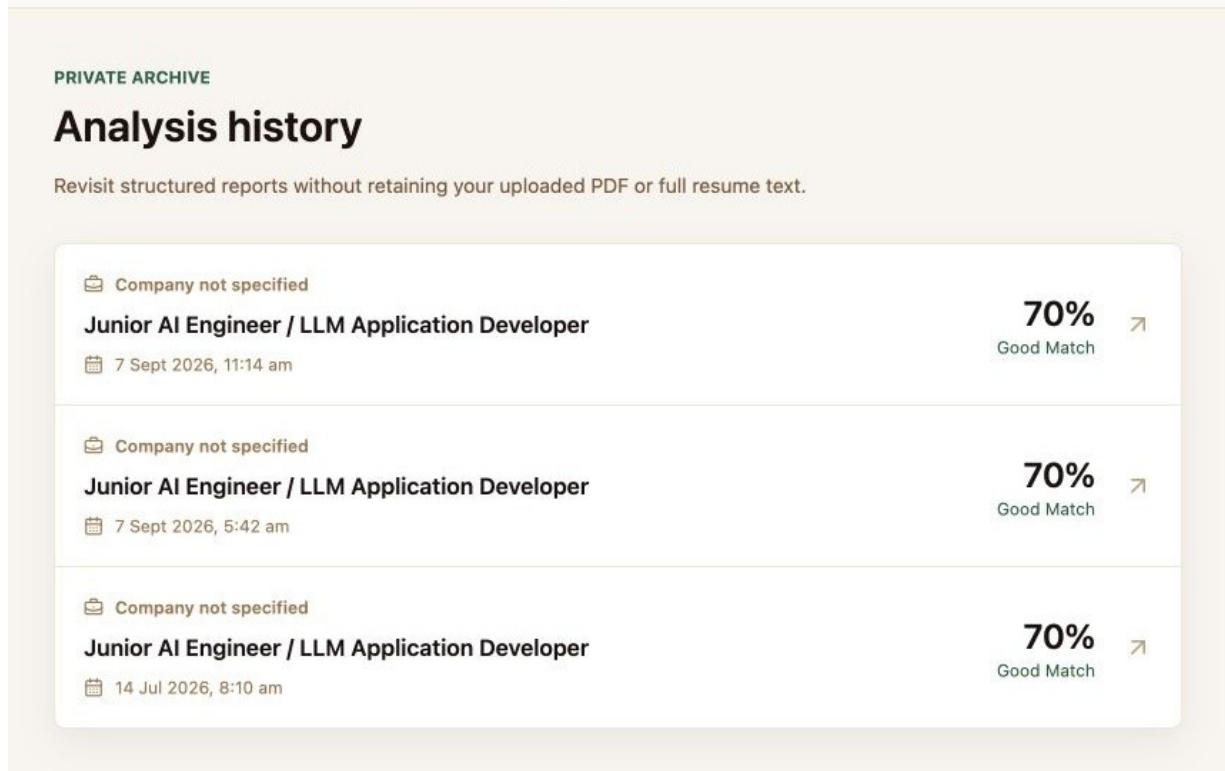


Figure 7. Owner-specific analysis history showing saved role, date, score and fit label. Earlier text-product build.

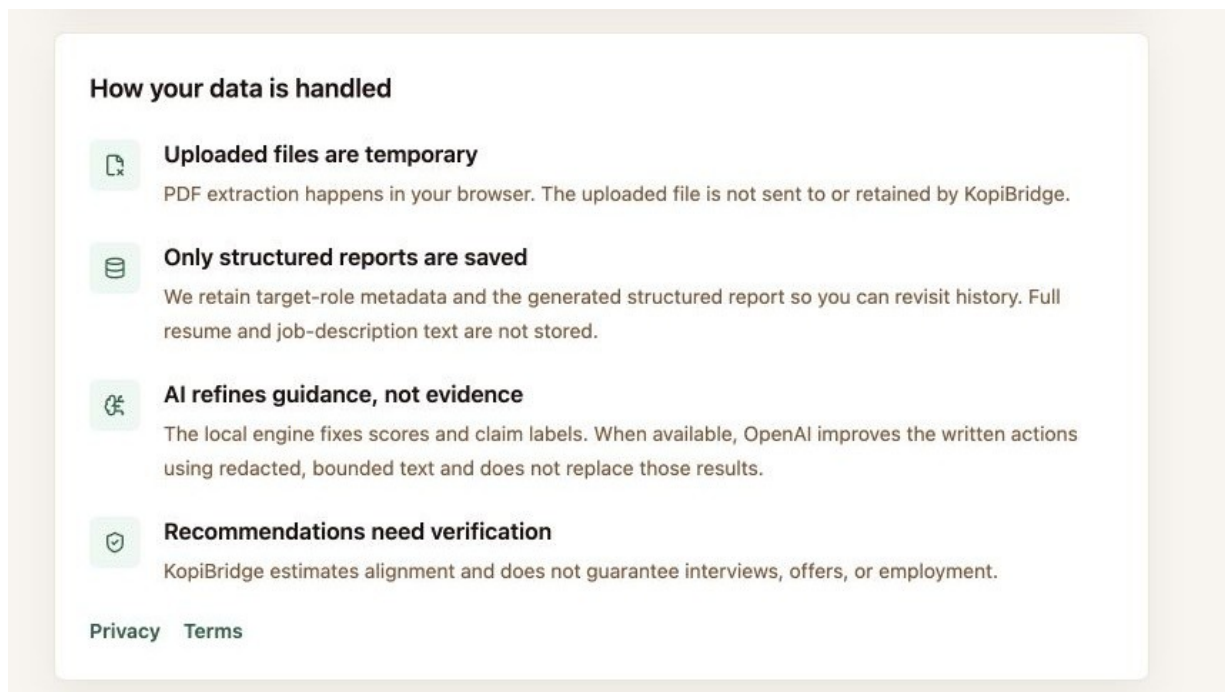


Figure 8. Privacy and trust information exposed to the signed-in user; personal account identifiers are cropped from this report figure. Earlier text-product build.

The stored record intentionally excludes the complete raw resume, complete job description and uploaded file bytes. It retains the target role, optional company, filename, status, structured report and small summary fields required for history. Users can delete their own reports, but the present prototype does not yet provide account-wide data export or deletion.

The Save Report control invokes `window.print()`. Print-specific CSS hides navigation, upload controls and buttons, switches to white surfaces and dark text, maintains report metadata and uses `break-inside: avoid` for compact report sections. This relies on the browser's Save as PDF facility and avoids creating an unsecured public export route.

4.8 User-interface implementation

The public page explains the product before registration and contains a realistic report preview, benefits, process, privacy summary and final call to action (Figure 9). Auth pages share a branded shell and state the daily allowance clearly (Figure 6). The signed-in dashboard foregrounds the next action, credit reset policy, evidence-first principles and recent reports (Figure 10), while an empty history provides a direct route to the first analysis.

The input workspace presents a visible three-stage flow and preserves values after recoverable failures. Figures 11 and 12 show the empty and sample-populated states; target role and company detection remain editable before spending a credit. Submission changes to a focused progress state (Figure 13) and duplicate submission is prevented. The report component renders the same typed data for a newly created report and a historical report. Responsive grids collapse to one column, as demonstrated by the 390-pixel captures in Figure 23.

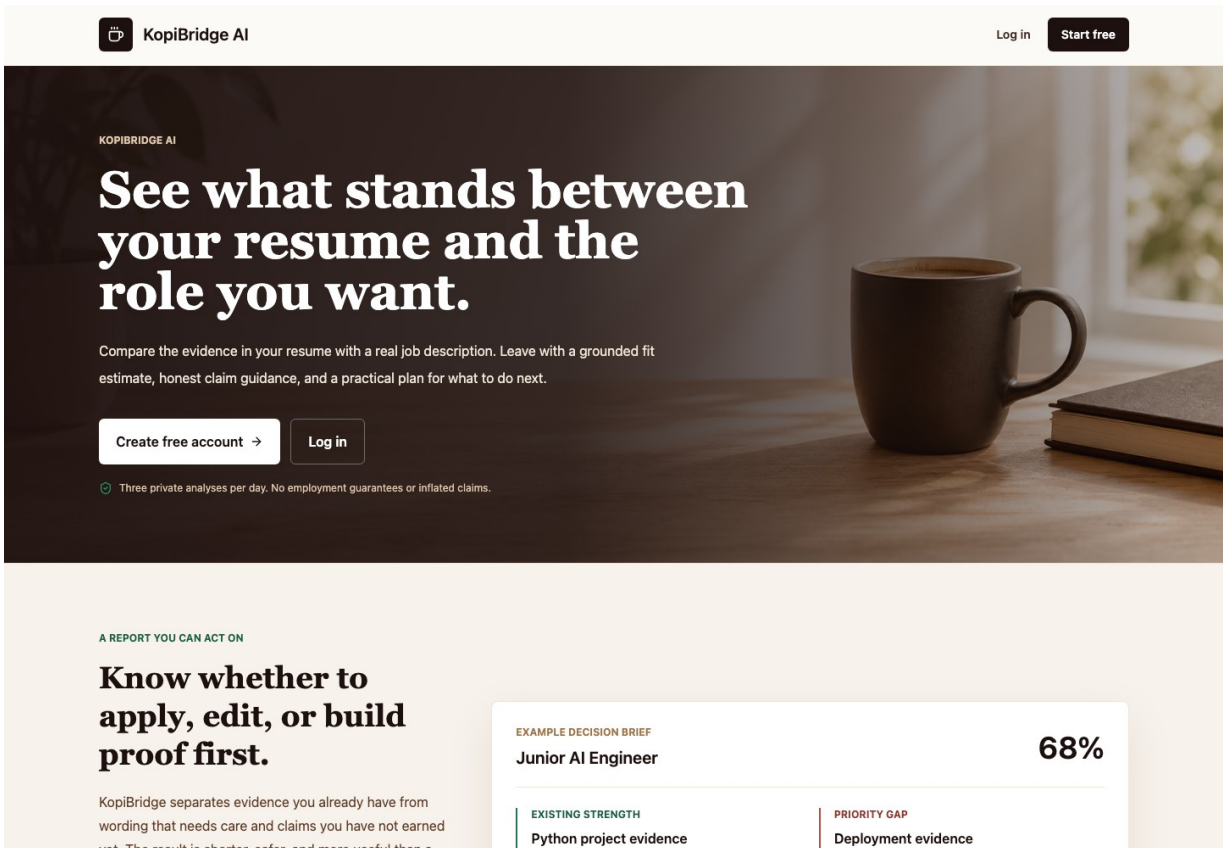


Figure 9. Public landing page communicates the product value and realistic output before registration. Earlier text-product build.

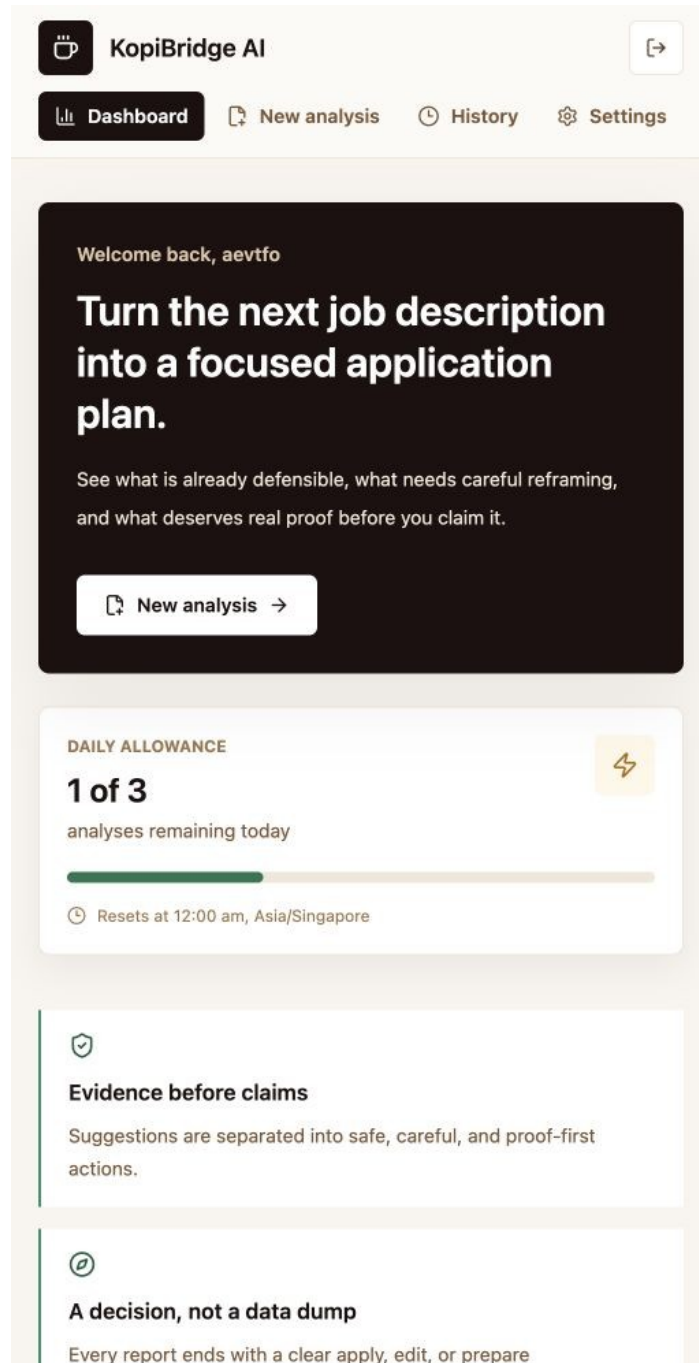


Figure 10. Signed-in dashboard with the primary action, daily allowance and evidence-first product principles. Earlier text-product build.



KopiBridge AI

2 of 3 analyses remaining
Resets midnight Singapore time

Dashboard

New analysis

History

Settings

NEW ANALYSIS

Compare evidence, not just keywords

Your PDF stays in the browser. Extracted text may be securely processed to refine the guidance, while only the completed report is saved to your private history.

2 of 3 remaining today

1 Resume evidence

2 Target role

3 Confirm and analyse



Resume source

Upload a text-based PDF, paste text, or load a sample.

Awaiting resume

Use Sample Resume Text



Choose a PDF resume

Client-side extraction, max 8MB

Extracted resume text

Hide

Upload a PDF or paste your resume text here.

Figure 11. Empty analysis workspace with the three-stage workflow, input guidance and sample fallback. Earlier text-product build.

**KopiBridge AI**

2 of 3 analyses remaining
Resets midnight Singapore time

[Dashboard](#)[New analysis](#)[History](#)[Settings](#)



Choose a PDF resume
Client-side extraction, max 8MB

Extracted resume textShow

**Target job description**

Paste the full role so requirements can be compared in context.

Job description ready

Load Demo Job Description

Target job description textShow

**Confirm the target**

Detection is a convenience. Correct it before spending a credit.

Target role

Company (optional)

Junior AI Engineer / LLM Application Develop

Company name

One accepted analysis uses one credit. Validation failures are free, and internal failures automatically restore the credit.

Analyse gap →

Figure 12. Sample-populated analysis state with editable role confirmation and a clear credit-cost message. Earlier text-product build.

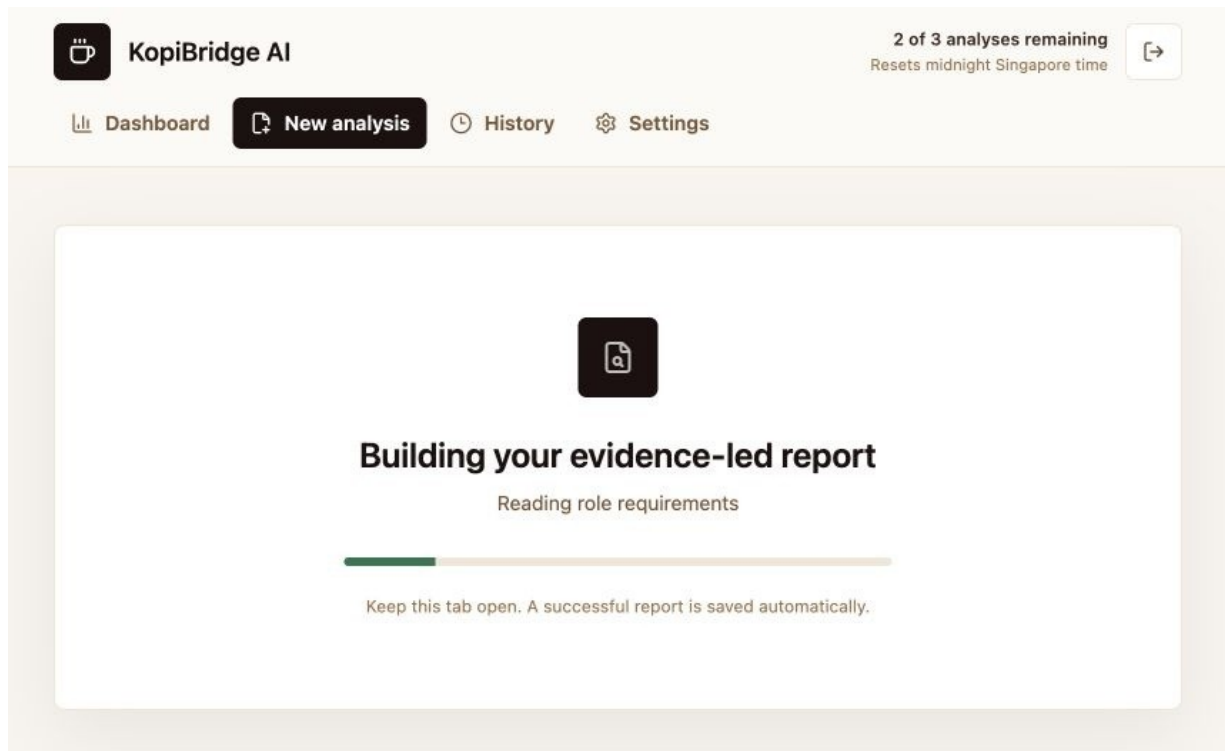


Figure 13. Focused progress state displayed while the server constructs and stores the report. Earlier text-product build.

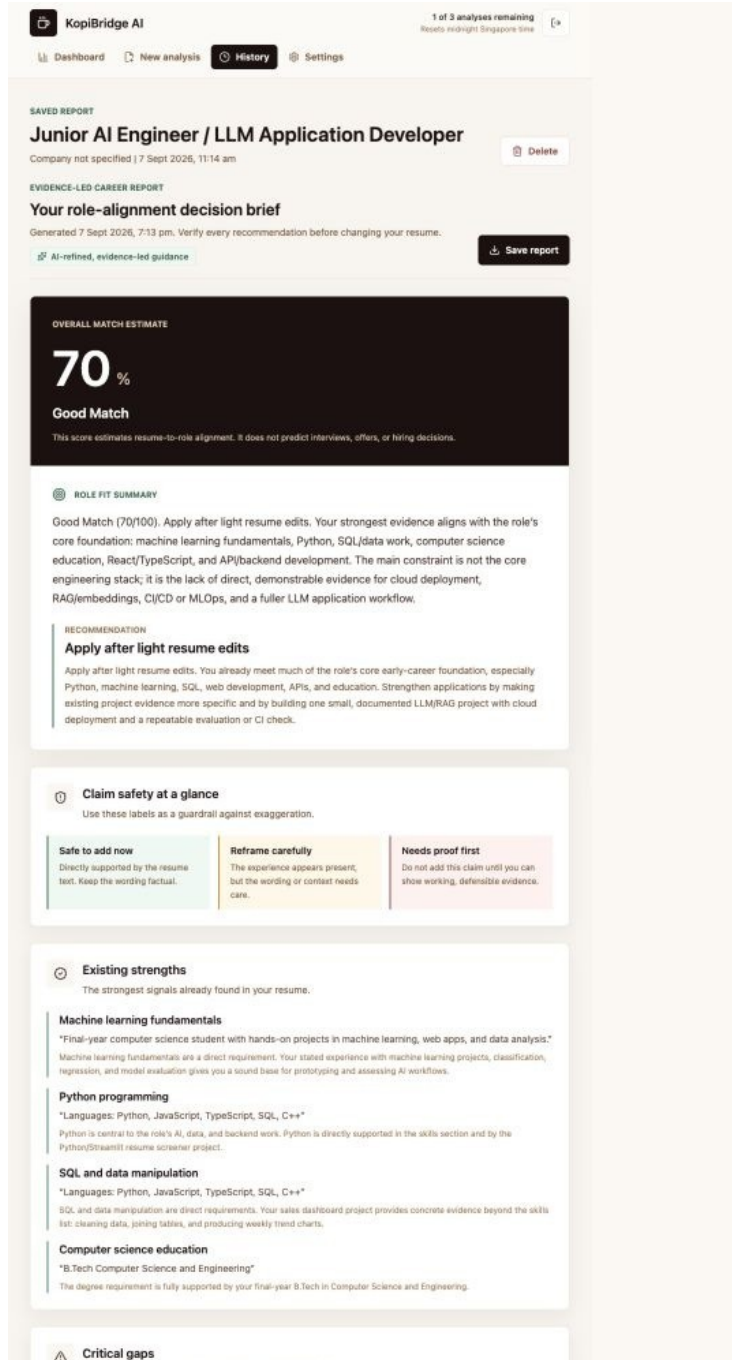


Figure 14. Executive summary showing the 70% estimate, role-fit reasoning and final apply/edit decision. Earlier text-product build.

Computer science education

"B.Tech Computer Science and Engineering"

The degree requirement is fully supported by your final-year B.Tech in Computer Science and Engineering.

Critical gaps

The three issues most likely to weaken this application.

1 Cloud platforms

Missing Skill | 3-7 Days

High

The target role requests exposure to AWS, GCP, or Azure. There is no direct cloud evidence on the resume, so naming a cloud platform now would not be defensible.

Proof step: Deploy one small existing or new application to a cloud platform, or complete a documented cloud walkthrough. Keep a working URL or deployment record, setup notes, environment-variable instructions, and a screenshot before adding the platform to the resume.

2 RAG / embeddings workflow

Missing Skill | 5-10 Days

High

The role specifically values LLM and retrieval workflows. No RAG or embeddings workflow is currently shown, and prompt testing alone is not enough to claim retrieval experience.

Proof step: Build a small RAG demo around a local knowledge base. Document the source documents, chunking approach, retrieval examples, one poor answer, and how you checked answer quality. Add RAG or vector-search language only after this evidence exists.

3 MLOps / CI-CD for ML

Missing Skill | 4-7 Days

High

CI/CD, MLOps, and production reliability are preferred qualifications. The resume has no direct evidence of automated checks, versioned evaluation, or delivery workflow.

Proof step: Add one lightweight automated workflow to a project, such as a GitHub Actions check that runs a test or evaluation command. Include the command, expected output, and a short README section explaining what the check validates.

Action roadmap

Start with tasks that create proof, then earn stronger resume wording.

Existing strengths

The strongest signals already found in your resume.

Machine learning fundamentals

"Final-year computer science student with hands-on projects in machine learning, web apps, and data analysis."

Machine learning fundamentals are a direct requirement. Your stated experience with machine learning projects, classification, regression, and model evaluation gives you a sound base for prototyping and assessing AI workflows.

Python programming

"Languages: Python, JavaScript, TypeScript, SQL, C++"

Python is central to the role's AI, data, and backend work. Python is directly supported in the skills section and by the Python/Streamlit resume screener project.

SQL and data manipulation

"Languages: Python, JavaScript, TypeScript, SQL, C++"

SQL and data manipulation are direct requirements. Your sales dashboard project provides concrete evidence beyond the skills list: cleaning data, joining tables, and producing weekly trend charts.

Computer science education

"B.Tech Computer Science and Engineering"

The degree requirement is fully supported by your final-year B.Tech in Computer Science and Engineering.

Critical gaps

The three issues most likely to weaken this application.

1 Cloud platforms

Missing Skill | 3-7 Days

High

The target role requests exposure to AWS, GCP, or Azure. There is no direct cloud evidence on the resume, so naming a cloud platform now would not be defensible.

Proof step: Deploy one small existing or new application to a cloud platform, or complete a documented cloud walkthrough. Keep a working URL or deployment record, setup notes, environment-variable instructions, and a screenshot before adding the platform to the resume.

2 RAG / embeddings workflow

Missing Skill | 5-10 Days

High

The role specifically values LLM and retrieval workflows. No RAG or embeddings workflow is currently shown, and prompt testing alone is not enough to claim retrieval experience.

Proof step: Build a small RAG demo around a local knowledge base. Document the source documents, chunking approach, retrieval examples, one poor answer, and how you checked answer quality. Add RAG or vector-search language only after this evidence exists.

Figure 15. Evidence-grounded strengths and the three highest-priority gaps with concrete proof steps. Earlier text-product build.

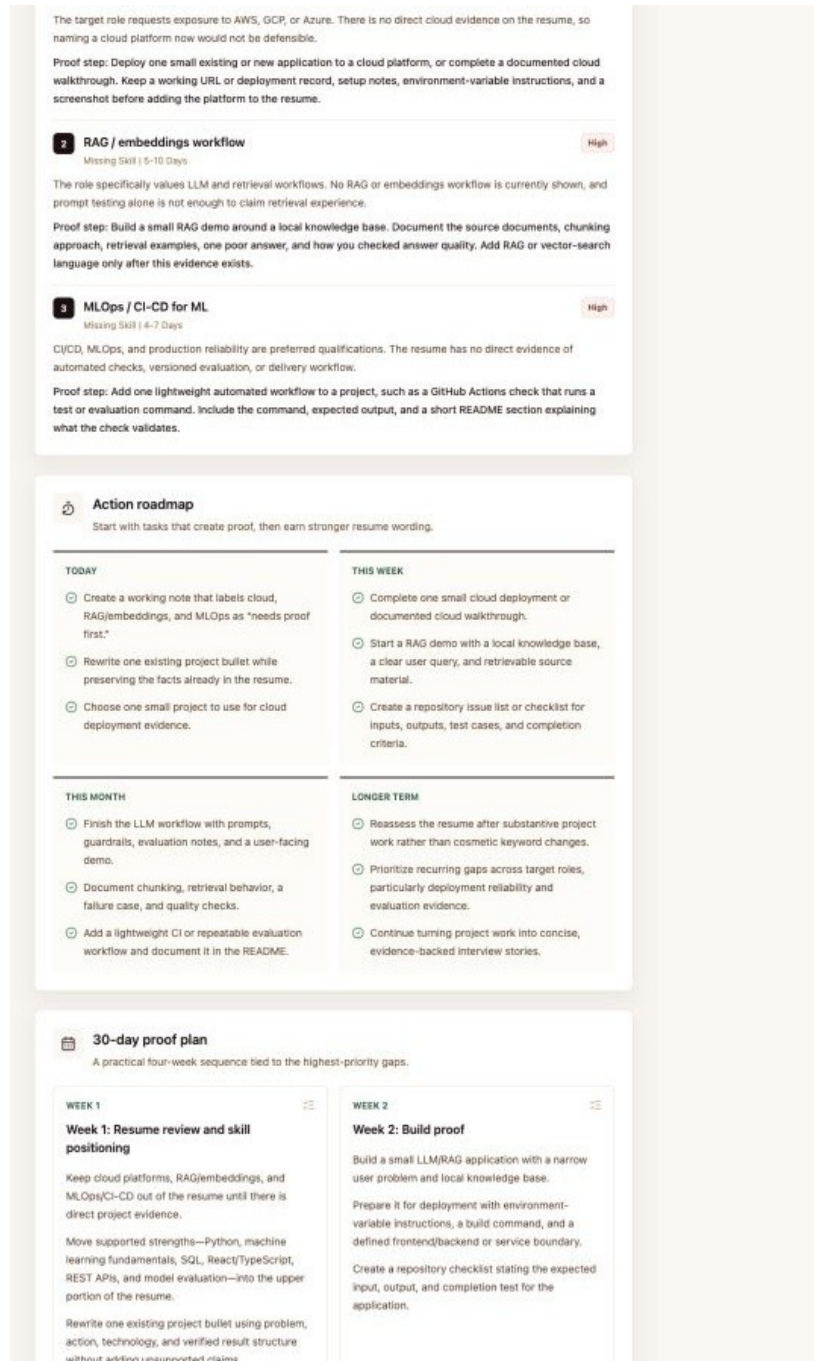


Figure 16. Time-bounded action roadmap generated from the current high-priority gaps. Earlier text-product build.

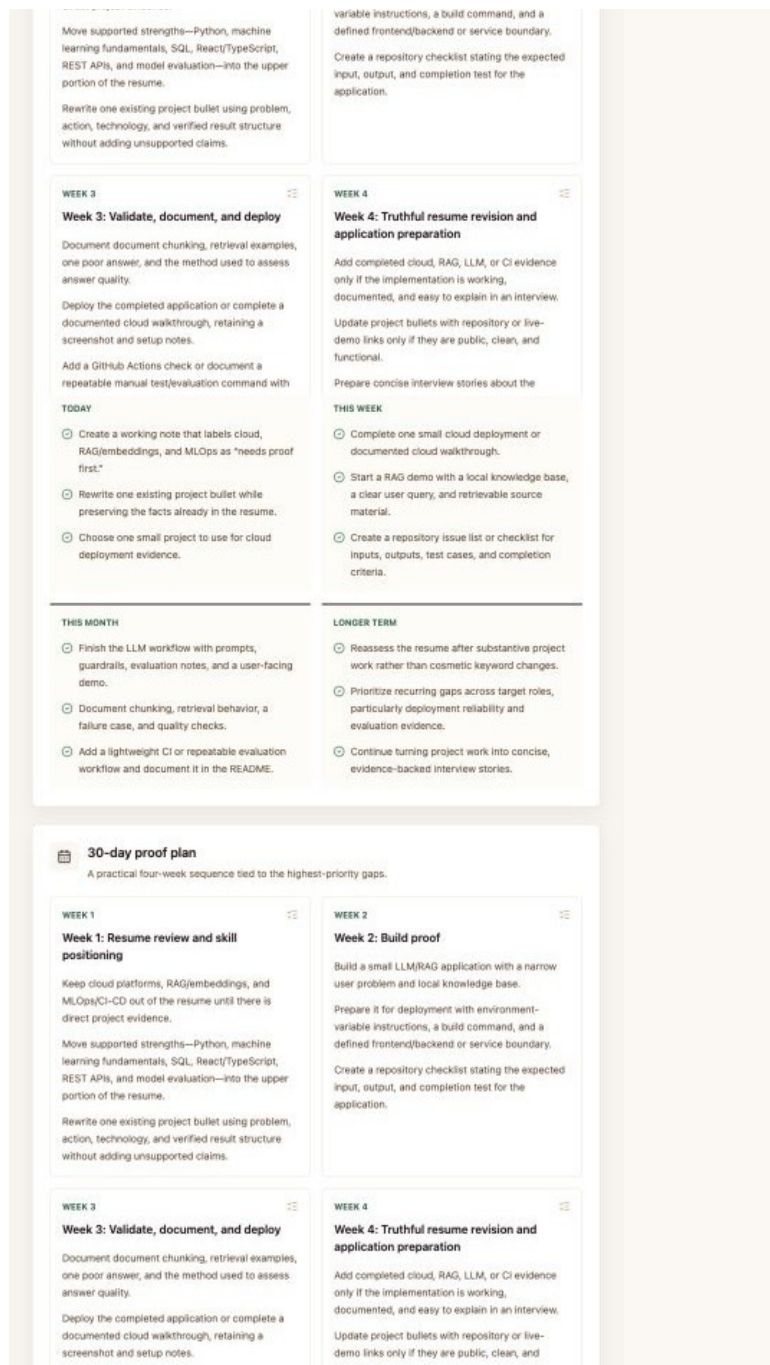


Figure 17. Four-week proof plan linking resume revision, a small build, validation, deployment and application preparation. Earlier text-product build.



Resume wording workshop

Conservative rewrites that expose missing evidence instead of inventing it.

CURRENT WORDING

Wrote SQL queries to clean sales data, join customer tables, and build weekly trend charts

Reframe carefully

Wrote SQL queries to clean sales data, join customer tables, and build weekly trend charts; presented findings and recommendations to a student project panel. [Add a verified result or scale only if you can evidence it.]

This combines the documented technical work with the documented presentation outcome, without introducing unverified metrics.

CURRENT WORDING

Built a Python and Streamlit app that compares resume text against role keywords and generates improvement suggestions

Reframe carefully

Built a Python and Streamlit app that compared resume text against role keywords and generated improvement suggestions using Pandas and Scikit-learn to calculate keyword coverage and visualize match categories. [Add a verified result or scale only if you can evidence it.]

This retains the original project facts while clarifying the technical implementation and the product behavior.



Interview preparation

Prepare evidence for strengths and honest language for gaps.

Five likely questions

- Tell us about a project where you built an AI-powered feature from idea to prototype.
- How did you evaluate whether your resume screener or chatbot behavior was useful?
- What prompt variations did you test in the Campus Helpdesk Chatbot, and what failure cases did you document?
- How would you design a small retrieval-based assistant for a set of documents?
- Describe the SQL work in the Sales Insights Dashboard and how you communicated the findings to others.

Evidence to prepare

A short walkthrough of the AI Resume Screener: input, keyword-coverage calculation, output categories, and improvement suggestions.

A specific example of a prompt variation and one common failure case from the Campus Helpdesk Chatbot documentation.

A concise account of the SQL cleaning, joins, weekly trend charts, and panel recommendations.

Monitoring and evaluation

EXPERIENCE

AI / Data: Pandas, NumPy, Scikit-learn, PyTorch, model evaluation, classification, regression

Reframe carefully

Partial evidence

MLOps / CI-CD for ML

TOOLS

No direct evidence found in the resume text.

Needs proof first

Missing evidence



Resume wording workshop

Conservative rewrites that expose missing evidence instead of inventing it.

CURRENT WORDING

Wrote SQL queries to clean sales data, join customer tables, and build weekly trend charts

Reframe carefully

Wrote SQL queries to clean sales data, join customer tables, and build weekly trend charts; presented findings and recommendations to a student project panel. [Add a verified result or scale only if you can evidence it.]

This combines the documented technical work with the documented presentation outcome, without introducing unverified metrics.

CURRENT WORDING

Built a Python and Streamlit app that compares resume text against role keywords and generates improvement suggestions

Reframe carefully

Built a Python and Streamlit app that compared resume text against role keywords and generated improvement suggestions using Pandas and Scikit-learn to calculate keyword coverage and visualize match categories. [Add a verified result or scale only if you can evidence it.]

Figure 20. Resume wording workshop separates supported rewrites from claims that require implementation evidence. Earlier text-product build.

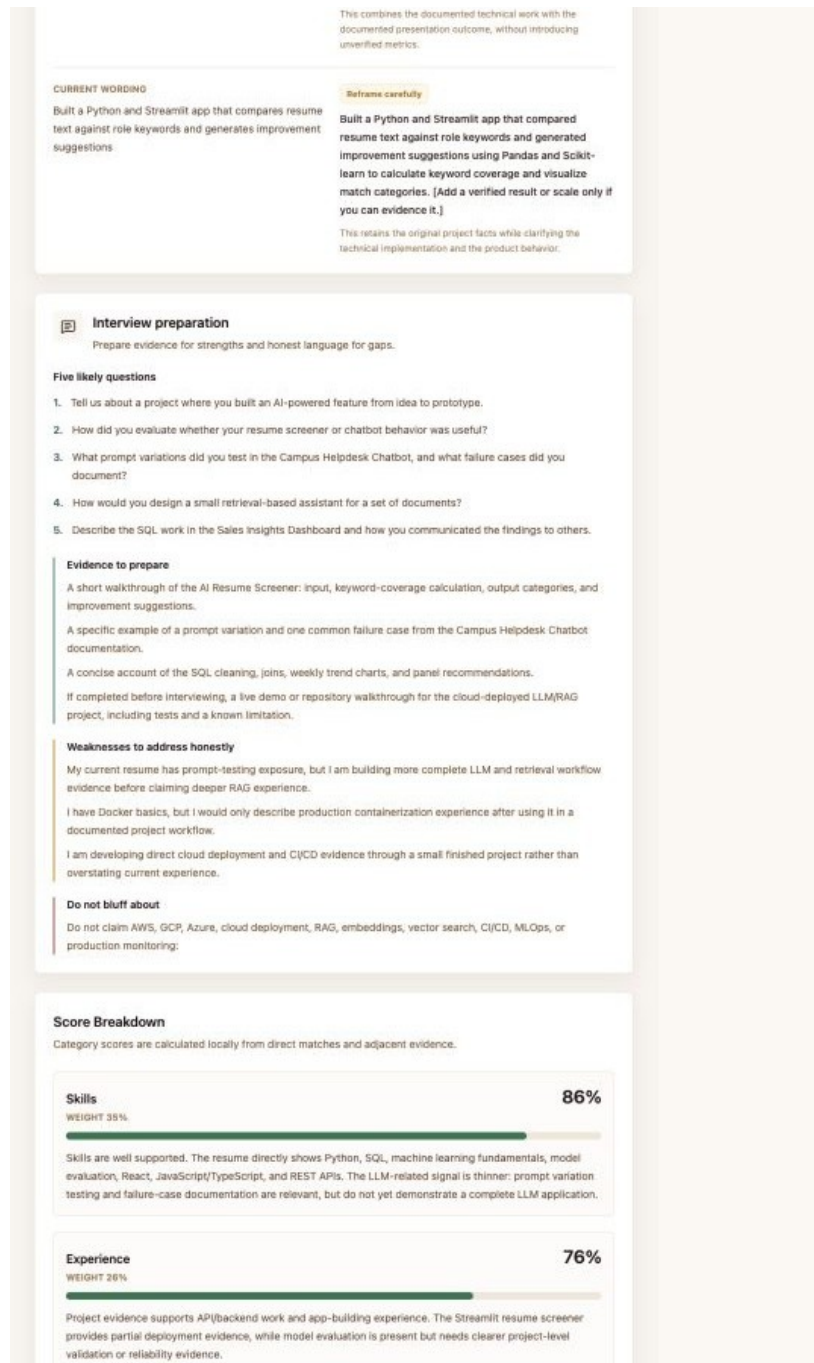


Figure 21. Role-specific interview preparation identifies likely questions, supporting evidence and areas not to bluff. Earlier text-product build.

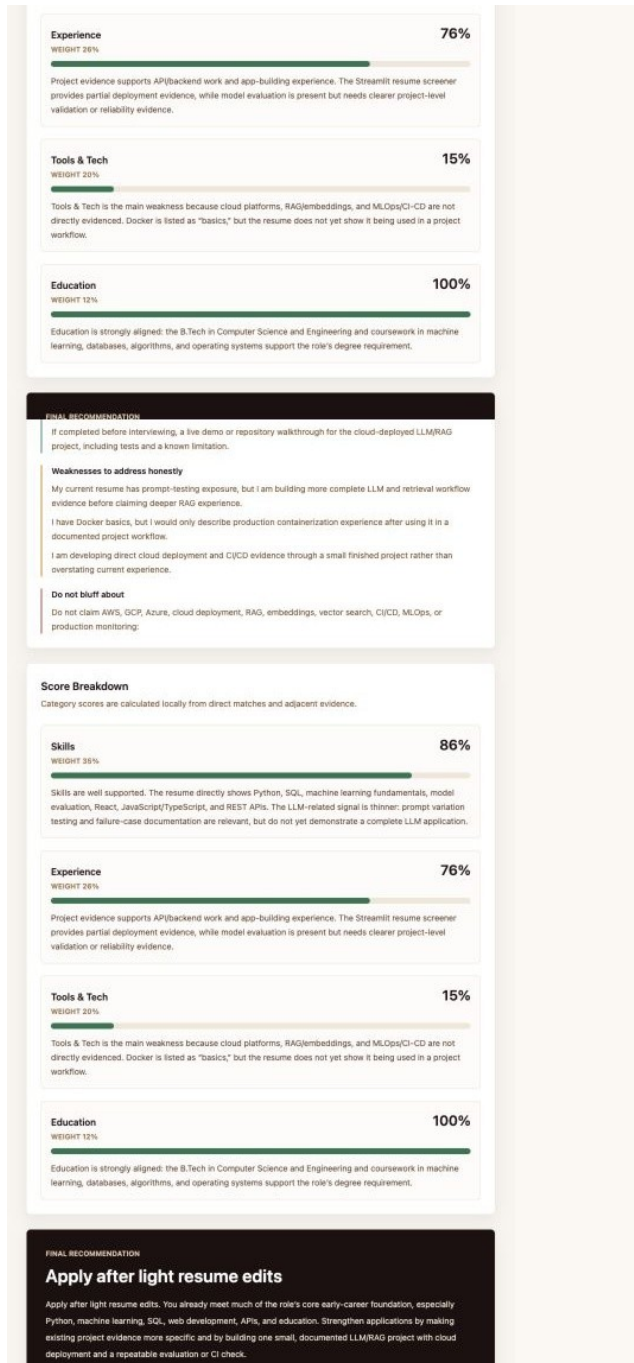
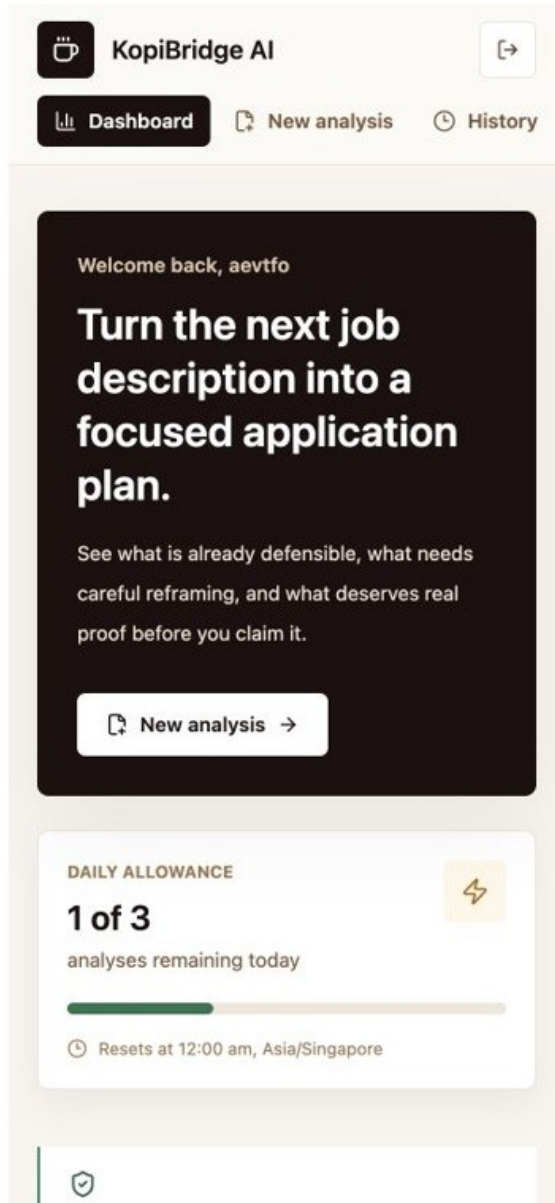


Figure 22. Category score breakdown and the evidence-based final recommendation at the end of the report. Earlier text-product build.

(a) Dashboard



(b) Report

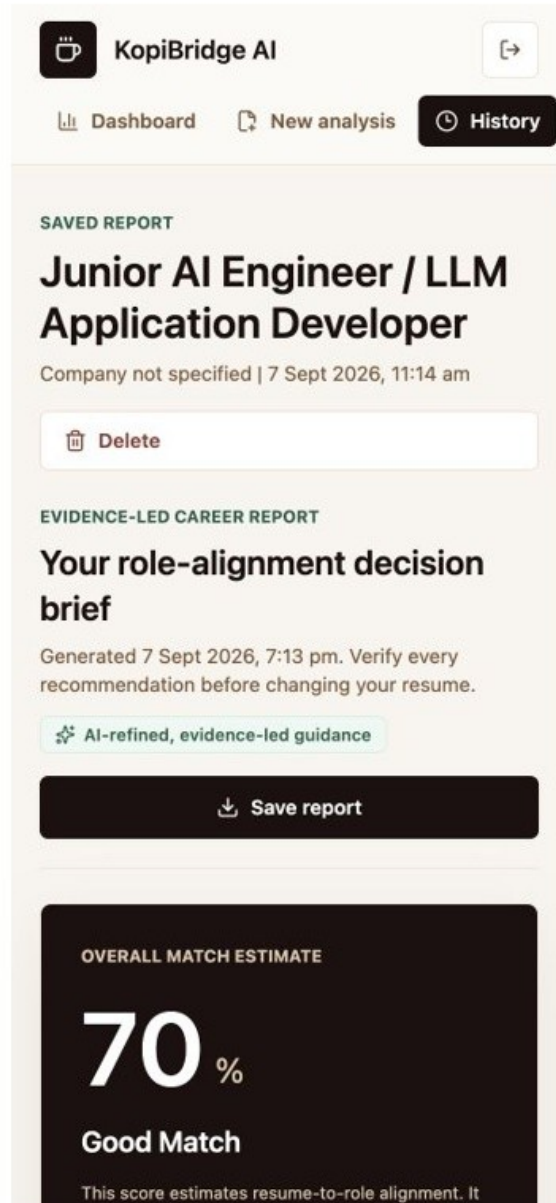


Figure 23. Mobile dashboard and report at 390 pixels, demonstrating single-column hierarchy without horizontal overflow. Earlier text-product build.

Figures 14-22 document the generated report from executive summary through final recommendation. The captured sample achieved a 70% Good Match: skills 86, experience 76, tools 15 and education 100. Its three highest gaps were cloud platforms, RAG/embeddings and MLOps/CI-CD, producing an Apply after light resume edits recommendation. The local engine fixed these scores and labels; the guarded model improved the explanations and proof-building steps. Figure 24 retains the compact deterministic fixture visualisation for comparison. This case demonstrates both useful strengths and deliberately conservative tool gaps; it is not an accuracy benchmark.



Highest proof-first gaps: cloud platforms • RAG / embeddings • MLOps / CI-CD

Final recommendation: Apply after light resume edits

Figure 24. Sample deterministic report output generated from the repository's fixed resume and job-description fixtures. Earlier text-product build.

4.9 Deployment and maintainability

The application is configured for Vercel's Next.js deployment. Public environment variables contain only the Supabase project URL and publishable key. Schema changes are recorded as SQL migrations, while secrets are excluded from the repository. The production build distinguishes static public pages from dynamic authenticated pages and route handlers.

The project remains one typed Next.js codebase. The additive migration `20260927203904_add_multimodal_career_workflow.sql` extends, rather than rewrites, the foundation. Model IDs are configurable server-side and checked for distinctness. Provider requests have bounded timeouts and no automatic retry multiplication. Reproducible synthetic fixtures, model-result JSON and a disposable-account browser runner support regression checks; the production app itself does not require an administrator key.

4.10 Scanned resumes and spoken interview practice

lib/openai-media.ts supplies server-only adapters for all three media operations. Image extraction uses the Responses API with a bounded text-and-warnings schema. lib/audio.ts decodes accepted audio locally and resamples it to mono 16kHz PCM WAV. The server checks canonical headers, byte counts, duration and non-silent energy before transcription. Audio is limited to ninety seconds; decoding and upload errors leave typed answers available. A selected report question is derived server-side, not accepted as arbitrary client authority.

InterviewPractice provides recording, upload, playback, consent, transcript correction and save states on the existing report page. The model receives the answer and report evidence, not permission to change scores. groundedFeedbackSchema constructs an enumeration of actual transcript spans for quotation fields; validateFeedbackQuotes then checks the returned quotations against the source. The rest of the feedback remains generated interpretation and still requires review. A timeout or invalid response stores a clearly labelled local preparation checklist instead of pretending that AI feedback succeeded.

The approved additive migration creates owner-only interview_practice and model_operations tables. save_interview_practice checks the reservation owner, parent report and selected question before saving. Report deletion cascades to feedback, while quota records remain until account deletion. The implementation uses media request schemas and streamed body limits, and blocks cross-site browser origins while accommodating Next.js localhost host normalisation. Hydration and audio object-URL lifecycle defects found during browser checks were fixed before the successful rerun.

Figures 25-30 document the final local workflow using synthetic content and live providers. They show consent, reviewed extraction, the resulting report, audio preparation, corrected transcription and saved feedback. The earlier Figures 6-23 document the established text-product interface; they are retained for coverage, not presented as evidence of the new media routes. The final mobile and credit-limit states are shown in Figures 31-32.

KopiBridge AI

3 of 3 analyses remaining

Resets midnight Singapore time

Dashboard

New analysis

History

Settings

NEW ANALYSIS

Compare evidence, not just keywords

Text-based PDFs are read on your device. Scanned pages can be sent to AI with your consent. Review the extracted text, then compare it with the role. Your completed report is saved privately.

3 of 3 remaining today

1 Resume evidence

2 Target role

3 Confirm and analyse

Resume source

Upload a PDF or resume image, paste text, or load a sample.

Awaiting resume

🕒

🔍 Use Sample Resume Text

📄

resume-scanned.pdf

PDF, PNG, JPEG, WebP - up to 8MB

Read scanned resume pages

Up to three pages will be sent to OpenAI for text extraction. KopiBridge does not save the images. Remove contact details before uploading if you prefer. Six attempts per day; no analysis credit is used.

☐ I agree to send these pages for AI extraction.

Read resume pages

Target job description

Paste the full role so requirements can be compared in context.

Awaiting target role

📄

Load Demo Job Description

Target job description text

Hide

Paste an AI-adjacent tech job description here. KopiBridge will compare your resume against the role requirements locally.

0 / 30,000 characters

Figure 25. Scanned PDF fallback requires explicit consent before provider processing.

41

KopiBridge AI

3 of 3 analyses remaining
Resets midnight Singapore time

[Dashboard](#) [New analysis](#) [History](#) [Settings](#)

NEW ANALYSIS

Compare evidence, not just keywords

Text-based PDFs are read on your device. Scanned pages can be sent to AI with your consent. Review the extracted text, then compare it with the role. Your completed report is saved privately.

3 of 3 remaining today

Resume evidence

2 Target role

3 Confirm and analyse

Resume source

Upload a PDF or resume image, paste text, or load a sample.

Resume text ready

🕒 Use Sample Resume Text

📄 resume-scanned.pdf

PDF, PNG, JPEG, WebP - up to 8MB

✕ Clear resume

AI-read text. Check every detail before continuing.

Extracted resume text

Hide

EARLY CAREER SOFTWARE DEVELOPER

EDUCATION

Target job description

Paste the full role so requirements can be compared in context.

Awaiting target role

📄 Load Demo Job Description

Target job description text

Hide

Paste an AI-adjacent tech job description here. KopiBridge will compare your resume against the role requirements locally.

0 / 30,000 characters

Figure 26. Vision output is editable and labelled for review before analysis.

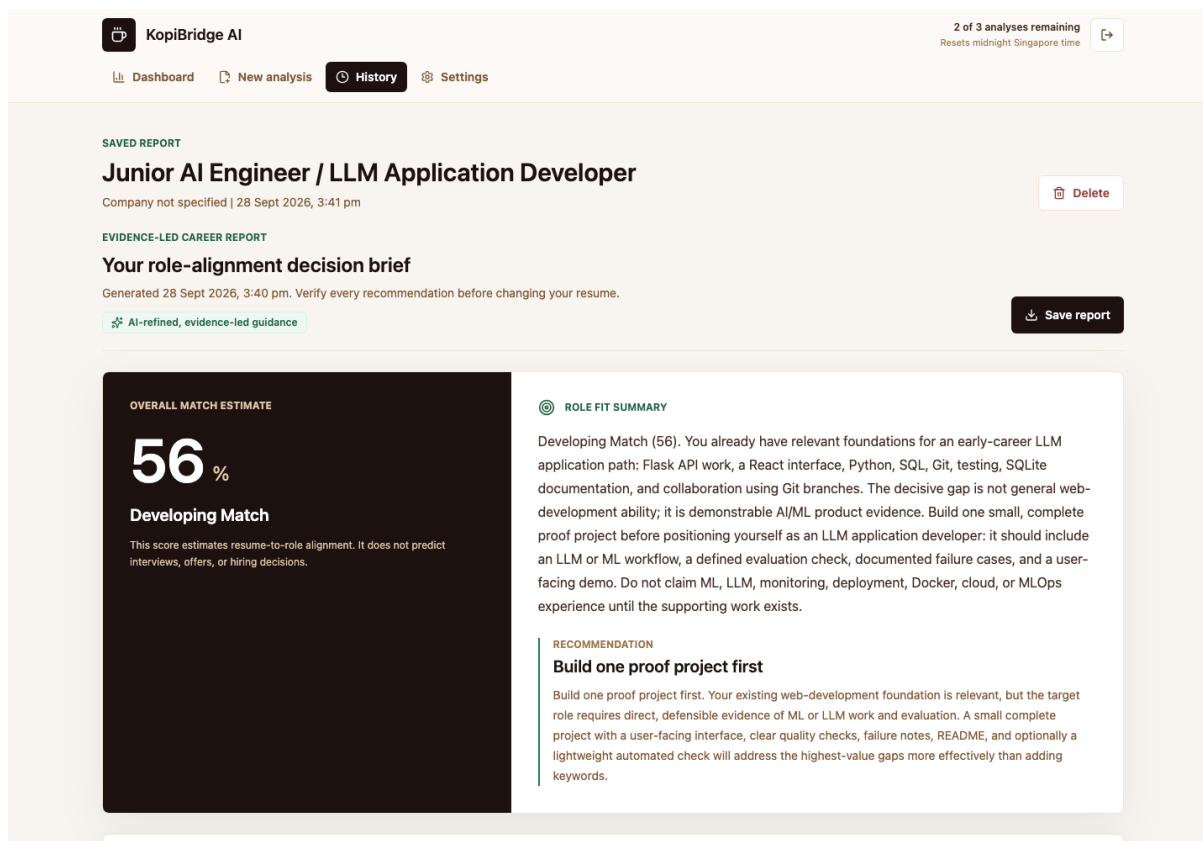


Figure 27. Actual report created from the scanned fixture, with one analysis credit spent.

KopiBridge AI

2 of 3 analyses remaining
Resets midnight Singapore time

[Dashboard](#) [New analysis](#) [History](#) [Settings](#)

FINAL RECOMMENDATION

Build one proof project first

Build one proof project first. Your existing web-development foundation is relevant, but the target role requires direct, defensible evidence of ML or LLM work and evaluation. A small complete project with a user-facing interface, clear quality checks, failure notes, README, and optionally a lightweight automated check will address the highest-value gaps more effectively than adding keywords.

➤ Today, define one narrowly scoped ML or LLM project and write its completion checklist: input, output, workflow, evaluation check, failure cases, README, and demo evidence.

Practise your evidence story

Use one question from your report to prepare a specific, honest answer. Spoken claims do not establish experience or change your resume assessment.

Practice question

Question 1

Walk me through the campus-events API and the decisions behind input validation and SQLite.

Record answer

Upload audio

Up to 90 seconds or 12MB. Your microphone starts only when you choose Record answer.

0:00 / 0:19

☒ I agree to send this audio and reviewed answer to OpenAI for transcription and feedback. KopiBridge saves only my reviewed text and feedback with this private report, not the audio.

Figure 28. Short interview audio can be previewed before transcription.

KopiBridge AI

2 of 3 analyses remaining
Resets midnight Singapore time

[Dashboard](#) [New analysis](#) [History](#) [Settings](#)

Question 1

Walk me through the campus-events API and the decisions behind input validation and SQLite.

Record answer

Upload audio

Up to 90 seconds or 12MB. Your microphone starts only when you choose Record answer.

0:00 / 0:19

☒ I agree to send this audio and reviewed answer to OpenAI for transcription and feedback. KopiBridge saves only my reviewed text and feedback with this private report, not the audio.

Transcribe answer

Review your answer

I built a Flask API for campus events. I added input validation and two unit tests using pytest. The events were stored in SQLite. I worked with two classmates using Git branches. I have not deployed the project to a cloud platform yet. I would prepare the repository and test results to explain my contribution.

☐ I have corrected the text and it represents my answer.

Save interview feedback →

No additional analysis credit. Six transcription attempts and six feedback attempts per day, resetting at midnight Singapore time. Failed processing attempts count towards these limits.

Transcript ready. Correct any mistakes, then confirm it represents your answer.

Figure 29. The applicant reviews the transcript before requesting feedback.

KopiBridge AI

2 of 3 analyses remaining
Resets midnight Singapore time

Dashboard New analysis History Settings

☐ I have corrected the text and it represents my answer.

Save interview feedback →

No additional analysis credit. Six transcription attempts and six feedback attempts per day, resetting at midnight Singapore time. Failed processing attempts count towards these limits.

Practice saved to this private report. Your original score and evidence labels are unchanged.

Saved interview feedback

28/09/2026

Walk me through the campus-events API and the decisions behind input validation and SQLite.

The answer establishes a credible Flask project, but it does not yet walk through the API or explain the validation and SQLite decisions. For this question, replace the project inventory with a brief request flow, the specific validation rules, why SQLite fit the project, and what the tests covered.

Your reviewed answer

What works

"I built a Flask API for campus events."

This is supported by the resume and gives a relevant backend starting point.

"I have not deployed the project to a cloud platform yet."

This is appropriately candid and avoids overstating deployment experience.

"I would prepare the repository and test results to explain my contribution."

Pointing to artifacts is a good instinct; a repository and test output can help substantiate your contribution.

Make it more specific

"I added input validation and two unit tests using pytest."

Explain the actual validation behavior: name the request fields checked, the invalid cases rejected, the HTTP response returned, and why those checks were needed. The pytest count and validation implementation are not supported by the supplied resume, so verify them with the repository before stating them.

"The events were stored in SQLite."

State the decision rationale: for example, whether SQLite was chosen because the project was a small local application needing a simple relational database. Only use reasons you can confirm from the project. SQLite use is not explicitly documented in the supplied resume evidence, so verify it.

"I worked with two classmates using Git branches."

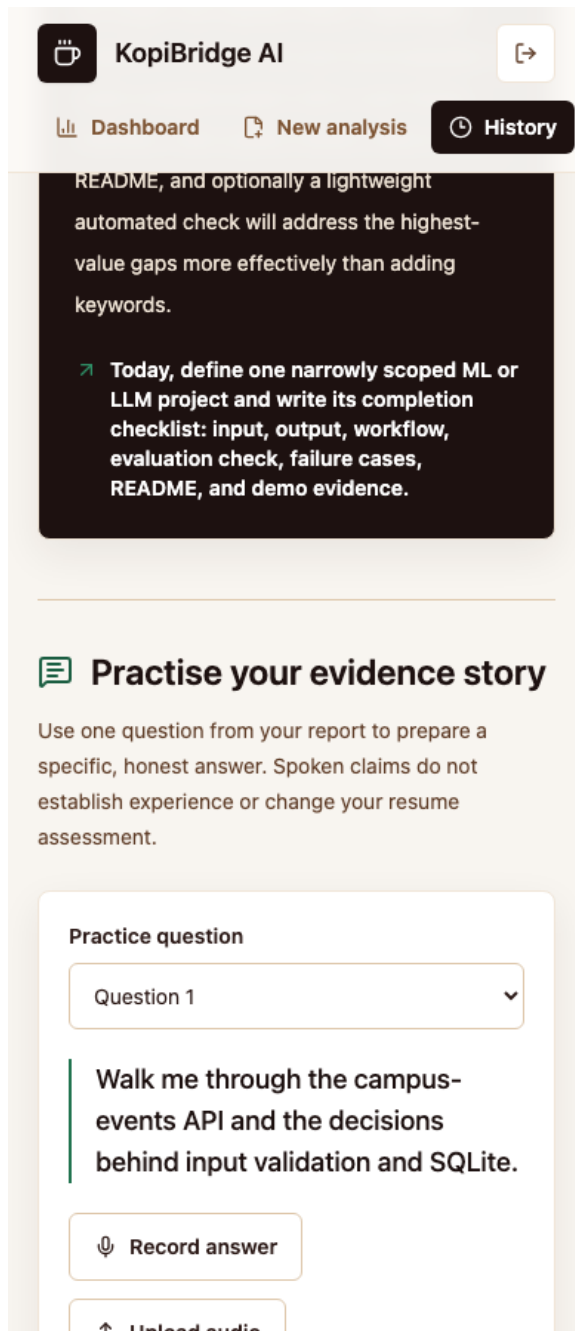


Figure 31. Final interview workflow at a 390-pixel viewport; no horizontal overflow.

KopiBridge AI 0 of 3 analyses remaining
Resets midnight Singapore time

Dashboard New analysis History Settings

NEW ANALYSIS

Compare evidence, not just keywords

Text-based PDFs are read on your device. Scanned pages can be sent to AI with your consent. Review the extracted text, then compare it with the role. Your completed report is saved privately. 0 of 3 remaining today

- Resume evidence
- Target role
- Confirm and analyse

Resume source

Upload a PDF or resume image, paste text, or load a sample.

Awaiting resume

Use Sample Resume Text

Choose a resume file

PDF, PNG, JPEG, WebP - up to 8MB

Extracted resume text

Hide

Upload a PDF or paste your resume text here.

Target job description

Paste the full role so requirements can be compared in context.

Awaiting target role

Load Demo Job Description

Target job description text

Hide

Paste an AI-adjacent tech job description here. KopiBridge will compare your resume against the role requirements locally.

0 / 30,000 characters

Figure 32. Daily analysis exhaustion produces a clear disabled state.

The provider interfaces follow OpenAI image and file-transcription documentation (OpenAI, n.d.-a; n.d.-b). API availability was checked through actual calls; provider model names identify pretrained services, not locally trained checkpoints.

5. Evaluation (1967/2500 words)

5.1 Evaluation strategy

Evaluation addresses model orchestration as well as the web product. Deterministic and schema tests cover invariant preservation; executable SQL tests cover ownership and quota transitions; live model comparisons measure extraction and grounding; and browser tests connect the modalities through a saved report. This combination addresses RQ1-RQ3 without treating a polished screen or one percentage as evidence of career effectiveness.

The current evaluation does not use interview outcomes as ground truth because they depend on labour-market conditions, applicant behaviour and employer decisions beyond the document. Nor does a passing test suite prove that career guidance is effective. Automated checks establish correctness for specified cases; manual production tests establish integration feasibility; screenshots establish that the intended states rendered; users and a labelled requirement-evidence corpus are still required to evaluate interpretation, semantic coverage and recommendation usefulness.

5.2 Test plan and acceptance criteria

Testing follows a risk-based plan (Table 3). Each case states its input, observable outcome and evidence level. Invalid requests should be rejected before processing, models must not change authoritative fields, and new media must not spend a full-analysis credit. Browser checks exercise the actual local application with live Supabase and OpenAI, using disposable synthetic accounts. Database tests run separately in PGLite; they are not labelled hosted stress tests.

The matrix distinguishes automated evidence, manual production evidence and remaining limitations. A Pass means the stated criterion was observed in the named run. Partial is used where a contract or prior controlled scenario passed but a stronger runtime test remains necessary. This prevents skipped credential-dependent tests or unperformed destructive fault injection from being counted as current passes.

Table 3. Risk-based functional, security and presentation test plan with observed results.

ID	Scenario and method	Acceptance criterion	Observed result	Status
TP01	Public browser UI	Desktop/mobile entry and validation	6 browser checks	Pass
TP02	Authentication	Login, persisted session, protected routes	Disposable accounts; signup email not rerun	Partial
TP03	Selectable PDF	Local editable extraction	Flask text recovered	Pass
TP04	Scanned PDF	Consent and real vision output	Live extraction to report	Pass

ID	Scenario and method	Acceptance criterion	Observed result	Status
TP05	Audio upload	Verified WAV and transcription	Live speech model	Pass
TP06	Interview feedback	Reviewed answer, source quotes, persistence	Live text model and reload	Pass
TP07	Credit and duplicate	One deduction for concurrent same-key requests	201 and 409; fourth request 429	Pass
TP08	Failure refund	One restoration for pending reservation	Executable SQL; no hosted provider fault injection	Partial
TP09	Day boundary	Singapore dates and independent daily limits	Unit and SQL fixtures	Pass
TP10	Owner isolation	Foreign report, feedback and delete denied	Second account 404	Pass
TP11	Deletion	Confirmation and owner-only deletion	Browser plus SQL cascade test	Pass
TP12	Grounding	Fixed fields preserved; unsupported quotes rejected	Contract tests and 26 model trials	Pass
TP13	Print	window.print called; printable report rendered	Chromium PDF; other browsers untested	Partial
TP14	Responsive	No overflow at 390, 768 and 1440 pixels	Authenticated browser checks	Pass
TP15	Usability benefit	Users understand claims and actionable next steps	Study proposed, not performed	Not tested

5.3 Automated test results

The current Vitest run passed 33 tests in eight files. Existing tests cover analysis, credit dates, validation and the narrative merge. New tests execute both SQL migrations in PGLite and check media quotas, duplicate keys, daily-date partitioning, owner isolation, allowed questions, save idempotency, deletion cascade and credit refund behaviour. HTTP tests verify authenticated identity, same-host origin handling and streamed size limits. Media tests reject malformed images, overlong or silent WAV data, injected identity fields and invented quotations. The constrained feedback schema accepts only transcript excerpts.

Six public Playwright checks passed across desktop and mobile profiles. A separate authenticated integration case passed against the local application and live services, using two disposable accounts removed afterwards. Table 4 summarises the final command checks. The earlier chart in Figure 33 is retained as historical text-product evidence and must not be read as the current test count.

Check	Result	Scope
Prettier	Pass	Repository formatting
Lint	Pass	ESLint and source security policy
TypeScript	Pass	Static checking
Vitest	33 tests; 8 files	Local logic, HTTP and executable SQL
Playwright	6 public + 1 integrated	Live providers and disposable accounts
Production build	Pass	Next.js 16 production compilation
npm audit	0 vulnerabilities	Installed dependencies

Table 4. Automated validation rerun for this report update.

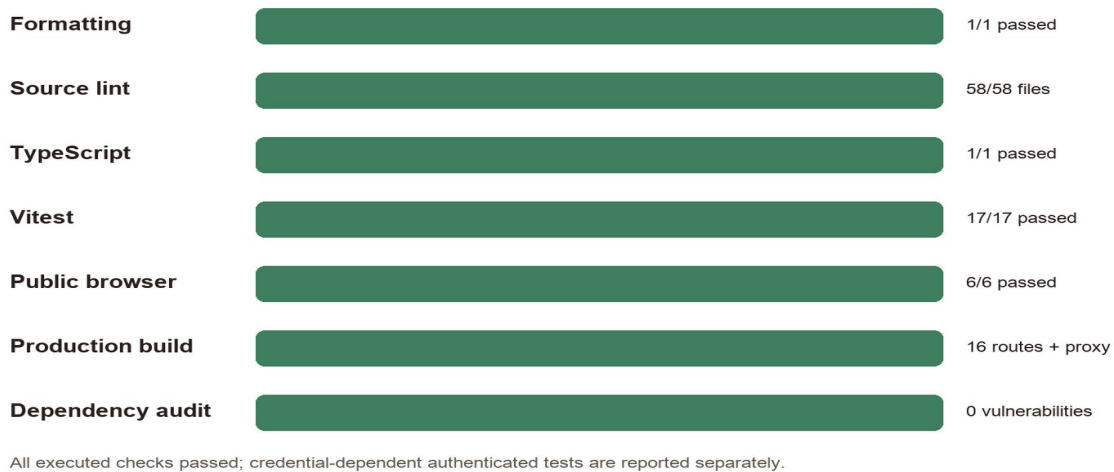


Figure 33. Earlier automated validation results; bar lengths show passed checks against executed checks in each suite. Earlier text-product build.

5.4 Authenticated end-to-end and security evaluation

The final authenticated case uploaded a selectable PDF, then an image-only PDF, obtained consented vision extraction and created a report. It uploaded a synthetic spoken answer, received a transcription, confirmed its text and saved model feedback. Reloading the report recovered the feedback. The daily balance fell from three to two after analysis and stayed at two

after transcription and feedback. A second user received an unavailable-report page and HTTP 404 for foreign feedback and deletion requests.

The same run submitted concurrent full-analysis requests with one idempotency key: exactly one returned HTTP 201 and the other returned HTTP 409. A third successful analysis exhausted the allowance; the fourth returned HTTP 429 and the UI disabled analysis. Owner deletion required confirmation and succeeded. Window.print invocation was observed through a test replacement, and Chromium produced a PDF for visual inspection. Screenshots are from the actual local application, not design mock-ups.

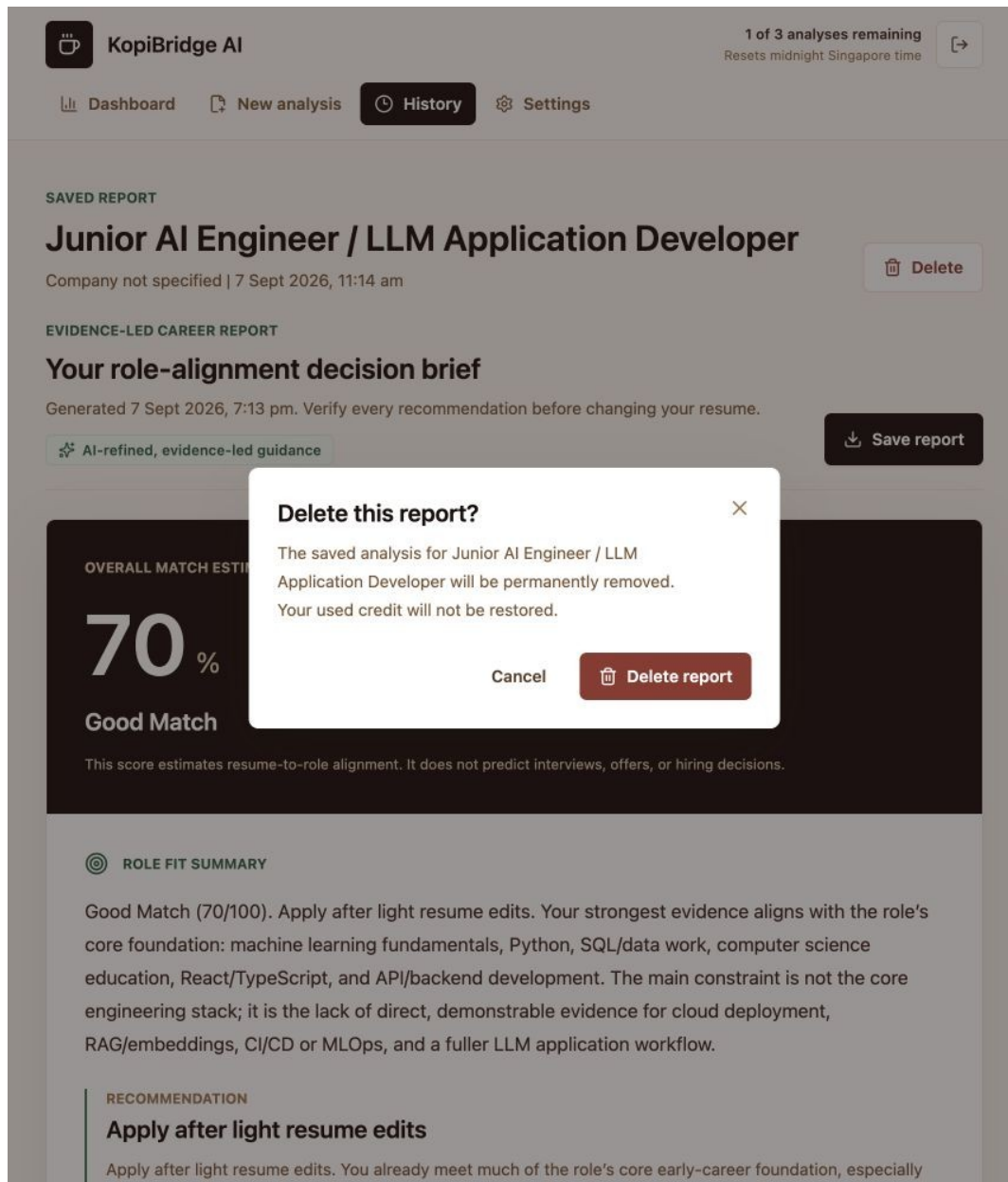


Figure 34. Destructive report deletion requires an explicit confirmation and states that spent credit is not restored. Earlier text-product build.

PGlite provides executable PostgreSQL semantics for the migrations but uses a single local connection, so its overlapping promises are not evidence of distributed concurrency. The hosted duplicate-request case gives narrower integration evidence, not a load benchmark. Supabase advisors reported intentional authenticated security-definer RPCs, disabled leaked-password protection and limited MFA options. Ownership checks and fixed search paths were reviewed, but these warnings and direct owner-callable RPCs mean the product should not be described as independently security-certified or its reports as tamper-proof credentials.

5.5 Functional and report evaluation

Earlier figures show the established 70% sample report. The new scanned fixture deliberately includes a negated statement: no Docker, cloud deployment or RAG project yet. The unchanged lexical matcher nevertheless assigned partial evidence to some of those terms. This is a substantive failure, not an OCR error: the image model reproduced the words correctly. Preserving deterministic fields limits model authority but also preserves deterministic mistakes. The report must therefore remain an inspectable aid; a future negation-aware evidence benchmark is higher priority than claiming calibrated accuracy.

The guarded text layer preserved all tested scores, evidence-map entries and gap severities for both candidate models. That is a contract result, not a proof of truthful prose. Inspection found that the smaller text candidate suggested inserting job-description terms too broadly, while feedback sometimes treated details absent from the compressed evidence context as unverified even when present in the full resume. These observations motivate explicit claim-level validation and better evidence selection. No precision, recall or employment-outcome statistic is inferred from the small fixtures.

The browser case found no horizontal overflow at widths 390, 768 and 1440 pixels and no captured console or page errors. The report remained readable with distinct claim labels. Print controls were hidden and a PDF was generated; Safari, unusual paper sizes and very long reports remain outside the observed scope. Microphone recording is implemented, but this automated end-to-end case tested the audio-upload path rather than a physical microphone.

5.6 Heuristic usability review

Against Nielsen's heuristics, visibility of system status is supported by workflow markers, loading progress, credit count and reset language. Error prevention is addressed through file checks, Zod validation, disabled submission, editable metadata and server-side reservation ordering. User control is supported by editable extracted text, report deletion and logout. Match with the real world is strongest in the evidence labels and time-based roadmap. Recognition rather than recall is supported by sample inputs and visible next actions.

The minimalist design is generally successful, but report length remains a tension. A user can understand the executive summary quickly, yet interview preparation, evidence, scores and rewrites create a long page. More collapsing could improve scanning, but hidden sections can weaken print output and discoverability. The current ordering is a reasonable compromise; task observation should determine whether users miss lower sections. Accessibility checks should

be expanded beyond semantic labels and focus styles to keyboard-only task completion and automated colour-contrast auditing.

5.7 Planned user study

A small formative study with approximately six to ten final-year students or recent graduates is proposed. Participants will use either a redacted personal resume or the sample, choose a real junior AI-adjacent job description and complete five tasks: create an analysis, state what the percentage means, identify the three top gaps, decide which claims are currently safe, and select the first roadmap action. Measures will include task success, completion time, assistance, observed errors, SUS, confidence and short interview responses.

Two comprehension items are essential. First, participants should answer whether a Needs proof first suggestion can be added immediately; the correct interpretation is no. Second, they should answer whether the match estimate predicts an interview; again, no. For action quality, participants can rate each top recommendation on relevance, specificity, feasibility and evidential value. Qualitative coding should identify confusing terminology, unrealistic effort estimates and missing evidence.

The study should avoid collecting unnecessary personal data. Participants can redact contact details, and raw resumes should not be retained in research notes. The report should record aggregate results and anonymised quotations only with consent. As a formative study, it will support design changes but not justify broad statistical generalisation.

5.8 Critical assessment and improvements

The strongest result is an integrated three-model path with preserved local evidence, private persistence and recoverable model failures. Quotation validation initially rejected five of six model responses; constraining allowed excerpts then produced six accepted reruns without loosening the verification rule. This is concrete iteration based on failure evidence. It does not guarantee that every interpretation of a correctly quoted sentence is sound.

The main weaknesses are lexical negation errors, compressed evidence context and evaluation breadth. Synthetic speech has limited accent and delivery diversity, and three resume layouts cannot represent real document noise. No participant usability scores or recruitment outcomes were collected. Hosted configuration still needs stronger account protections, and a future design should prevent clients from directly supplying apparently model-produced report content through owner-callable write RPCs.

Next priorities are a human-labelled evidence set with negation cases, a consented speech/document corpus, repeated candidate-model runs and the formative study described above. Operational work includes recording-permission tests, multi-browser print checks, least-privilege server-only finalisation and account-wide deletion. These improvements follow observed limitations rather than adding unrelated features.

5.9 Model comparison and observed iteration

The reproducible dataset contains three synthetic resume layouts (clean, two-column and low-contrast) and three short synthetic answers (specific, unsupported-claim and noisy). macOS speech synthesis supplies a known reference; fixed-seed noise adds one controlled perturbation. Every candidate sees the same inputs. Word error rate is (substitutions + deletions + insertions) divided by reference words after lowercasing and removing punctuation. This measures content order as well as recognition, but misses punctuation and semantic harm. Latencies are single observations, not service-level estimates.

Table 5. Exploratory live extraction comparison on synthetic fixtures.

Task	Candidate	Cases	Mean WER	Mean latency
vision	gpt-4.1-mini	3	0.00%	4.01s
vision	gpt-4o-mini	3	6.22%	3.04s
audio	gpt-4o-mini-transcribe	3	0.68%	1.90s
audio	whisper-1	3	0.68%	2.10s

The selected vision model reproduced all three normalised references. gpt-4o-mini had 18.67% WER on the two-column case and zero on the others, so gpt-4.1-mini was retained for this workflow despite slower average latency. Both transcription candidates tied at 0%, 2.04% and 0% WER; the selected candidate was slightly faster on average. This is weak selection evidence from one synthetic voice, not proof that Whisper is generally inferior. Both text candidates preserved fixed fields, but the smaller model was not promoted because that invariant alone did not establish safer advice.

There were 26 recorded provider trials: six vision, six transcription, two report-guidance, six initial interview-feedback and six revised-feedback calls. Five initial interview responses failed exact-quotation validation. The application rejected them. Constraining quotations to source-span enums yielded six accepted reruns. This demonstrates a useful engineering correction, not zero hallucination: quoted evidence can still be misinterpreted and unseen inputs can fail. Token usage is retained where returned, but no unmeasured monetary cost or statistical significance is claimed.

6. Conclusion (346/1000 words)

KopiBridge AI implements CM3020 Project Idea 1, section 4.1, through three distinct pretrained models handling images, text and audio within one career-preparation workflow. Scanned documents become reviewed resume text; a deterministic report anchors language guidance; and a reviewed interview transcript receives private feedback tied to that report. The original account, credit, history and print functions remain part of the integrated product.

The contribution is the orchestration and evidence policy rather than a new foundation model. Explicit review points, constrained response schemas and immutable local fields reduce the opportunity for generated prose to become unsupported authority. The quotation redesign shows how evaluation can lead to a concrete implementation improvement. At the same time, a preserved heuristic can still be wrong, so inspectability must not be confused with semantic correctness.

The final evidence includes 33 passing automated tests, six passing public browser checks, one passing authenticated end-to-end workflow and 26 recorded live model trials. The selected vision model had zero normalised word error on three synthetic layouts; the selected speech model had zero error on two answers and one word error on the third. Both text candidates preserved fixed assessment fields. Five initial interview outputs were rejected for quotation mismatch, followed by six valid constrained reruns. These small results support technical feasibility, not general model superiority or a measured improvement in employability.

The multimodal code is validated locally against live services and its approved database migration is applied. Publishing that extension remains separate from implementation. The public repository must contain the submitted revision and remain accessible until results are issued. Further research should test real users, diverse speech and document layouts, claim-level correctness and the effect of completed proof tasks. No unperformed user study is presented as a finding.

More broadly, the project argues that responsible career technology should help users build truthful evidence rather than optimise appearances alone. KopiBridge AI does not promise employment and should not decide whether a person is capable. Its value lies in making a document comparison inspectable, converting uncertainty into proportionate action, and preserving the user's responsibility for what they ultimately claim.

References

- Amershi, S. et al. (2019) Guidelines for Human-AI Interaction. Proceedings of CHI 2019. doi:10.1145/3290605.3300233. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
- Bogen, M. and Rieke, A. (2018) Help Wanted: An Examination of Hiring Algorithms, Equity, and Bias. Upturn. Available at: <https://www.upturn.org/work/help-wanted/>
- Brooke, J. (1996) 'SUS: A quick and dirty usability scale', in Jordan, P.W., Thomas, B., Weerdmeester, B.A. and McClelland, I.L. (eds.) Usability Evaluation in Industry. London: Taylor & Francis.
- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', Proceedings of NAACL-HLT 2019, pp. 4171-4186. doi: 10.18653/v1/N19-1423. Available at: <https://aclanthology.org/N19-1423/>
- Doshi-Velez, F. and Kim, B. (2017) 'Towards a rigorous science of interpretable machine learning', arXiv:1702.08608. Available at: <https://arxiv.org/abs/1702.08608>
- European Commission (n.d.) What is ESCO? Available at: <https://esco.ec.europa.eu/en/about-esco/what-esco>
- Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F. and Pedreschi, D. (2018) 'A survey of methods for explaining black box models', ACM Computing Surveys, 51(5), Article 93. doi: 10.1145/3236009.
- ISO (2019) ISO 9241-210:2019 Ergonomics of human-system interaction - Part 210: Human-centred design for interactive systems. Available at: <https://www.iso.org/standard/77520.html>
- Kim, G. et al. (2022) OCR-free Document Understanding Transformer. Proceedings of ECCV 2022. <https://arxiv.org/abs/2111.15664>
- Mozilla (n.d.) PDF.js. Available at: <https://mozilla.github.io/pdf.js/>
- National Center for ONET Development (n.d.) The ONET Content Model. Available at: <https://www.onetcenter.org/content.html>
- Nielsen, J. (1994) 10 Usability Heuristics for User Interface Design. Nielsen Norman Group. Available at: <https://www.nngroup.com/articles/ten-usability-heuristics/>
- OpenAI (n.d.-a) Images and vision. <https://developers.openai.com/api/docs/guides/images-vision> (Accessed 28 September 2026).
- OpenAI (n.d.-b) File transcription. <https://developers.openai.com/api/docs/guides/speech-to-text> (Accessed 28 September 2026).
- OpenAI (n.d.-c) Structured model outputs. <https://developers.openai.com/api/docs/guides/structured-outputs> (Accessed 28 September 2026).
- OWASP (n.d.) Input Validation Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. and Sutskever, I. (2023) Robust Speech Recognition via Large-Scale Weak Supervision. Proceedings of ICML, PMLR 202, pp. 28492-28518. <https://proceedings.mlr.press/v202/radford23a.html>

Raghavan, M., Barocas, S., Kleinberg, J. and Levy, K. (2020) 'Mitigating bias in algorithmic hiring: Evaluating claims and practices', Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency, pp. 469-481. doi: 10.1145/3351095.3372828.

Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: Sentence embeddings using Siamese BERT-networks', Proceedings of EMNLP-IJCNLP 2019, pp. 3982-3992. doi: 10.18653/v1/D19-1410. Available at: <https://aclanthology.org/D19-1410/>

Supabase (n.d.-a) Server-Side Rendering. Available at: <https://supabase.com/docs/guides/auth/server-side>

Supabase (n.d.-b) Row Level Security. Available at: <https://supabase.com/docs/guides/database/postgres/row-level-security>

University of London (n.d.) Final Project Templates. CM3020 Artificial Intelligence, Project Idea 1, section 4.1, pp. 19-21. Course project brief supplied with the project.